

АННОТАЦИЯ

Выпускная квалификационная работа выполнена в форме дипломной работы на тему: «Отказоустойчивая информационная система, основанная на микросервисной архитектуре».

Объем дипломной работы – 66 страниц, на которых размещены 33 рисунка, 8 таблиц, 8 листингов и 2 формулы. Использовано 23 источника.

Ключевые слова: отказоустойчивость, микросервисная архитектура, контейнеры, оркестраторы контейнеров, CI/CD-конвейеры, стратегии развертывания.

Объектом исследования дипломной работы являются микросервисные информационные приложения.

Предметом исследования является построение отказоустойчивой информационной системы с применением инструментов контейнеризации и оркестрации контейнеров.

Дипломная работа состоит из введения, трех глав и заключения.

Во введении раскрывается актуальность темы, обозначаются проблемы, цель и задачи, определяются объект и предмет исследования дипломной работы, указывается практическая значимость.

Первая глава посвящена анализу принципов и методов построения отказоустойчивых информационных систем на инфраструктурном и архитектурном уровнях, анализу лучших практик, более детальному анализу актуальности темы, также здесь проводится сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем и рассматривается разработка архитектуры отказоустойчивой микросервисной информационной системы.

Вторая глава посвящена практической реализации прототипа отказоустойчивой информационной системы на микросервисной архитектуре и построению CI-конвейера, автоматизирующего процессы сборки и публикации Docker образов.

В третьей главе проводится тестирование прототипа разработанной информационной системы, в ходе которого продемонстрирована высокая эффективность современных архитектурных решений обеспечения отказоустойчивости.

Заключение содержит выводы по работе, обобщает результаты исследования и указывает на перспективы дальнейшего развития разработанного прототипа информационной системы.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	8
ГЛАВА 1. АНАЛИЗ ПРИНЦИПОВ И МЕТОДОВ ПОСТРОЕНИЯ ОТКАЗОУСТОЙЧИВЫХ ИНФОРМАЦИОННЫХ СИСТЕМ.....	10
1.1 Анализ актуальности микросервисной архитектуры для построения отказоустойчивых информационных систем	10
1.2 Планирование информационной системы	11
1.3 Отказоустойчивость на инфраструктурном уровне	12
1.4 Отказоустойчивость на архитектурном уровне	13
1.4.1 Основа архитектурной устойчивости	13
1.4.2 Монолит или микросервисная архитектура	13
1.4.3 Виртуальные машины и контейнеры	16
1.4.4 Оркестраторы контейнеров.....	18
1.4.5 Масштабирование базы данных.....	19
1.4.6 Балансировка нагрузки.....	21
1.5 Анализ лучших практик построения отказоустойчивых информационных систем	22
1.5.1 DevOps	22
1.5.2 Continuous Integration и Continuous Delivery / Deployment.....	23
1.5.3 Среды развертывания	24
1.5.4 Стратегии развертывания приложений	25
1.5.5 Мониторинг	28
1.5.6 Восстановление после сбоев	28
1.6 Сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем	29

1.6.1	Контейнерные платформы	29
1.6.2	Оркестраторы контейнеров.....	30
1.7	Разработка архитектуры отказоустойчивой микросервисной информационной системы	32
1.8	Постановка задачи	32
	Вывод по 1 главе	33
ГЛАВА 2. РЕАЛИЗАЦИЯ ПРОТОТИПА ОТКАЗОУСТОЙЧИВОЙ МИКРОСЕРВИСНОЙ СИСТЕМЫ.....		35
2.1	Структура проекта	35
2.2	Микросервисы	35
2.2.1	Фронтенд: UI.....	35
2.2.2	Бэкенд: шифрование и дешифрование ГОСТ Р 34.12-2015.....	36
2.2.3	Бэкенд: вспомогательные инструменты	37
2.3	Docker образы	37
2.3.1	Микросервис фронтенда.....	37
2.3.2	Микросервис шифрования и дешифрования	38
2.3.3	Микросервис инструментов	38
2.3.4	Использование хеш-сумм и минимальных базовых образов	38
2.4	Swagger-документация микросервисов.....	39
2.5	Валидация данных	41
2.6	Дополнение нулями для алгоритма шифрования «Кузнечик»	44
2.7	Краткое описание API микросервисов бэкенда.....	46
2.8	CI-конвейер в Jenkins.....	47
2.9	Kubernetes манифесты.....	50

Вывод по 2 главе	54
ГЛАВА 3. ТЕСТИРОВАНИЕ ОТКАЗОУСТОЙЧИВОСТИ ПРОТОТИПА	55
3.1 Проверка работоспособности.....	55
3.2 Недоступность микросервиса	56
3.3 Масштабирование микросервисов.....	57
3.4 Обновление системы	58
Вывод по 3 главе	59
ЗАКЛЮЧЕНИЕ.....	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	64

ВВЕДЕНИЕ

В современном мире, где бизнес все больше зависит от ИТ-технологий, вопросы информационной безопасности приобретают все большую значимость. Информационная безопасность – это различные меры по защите информации от посторонних лиц [1].

В концепции информационной безопасности выделяют три принципа:

- конфиденциальность – доступ к информации есть только у того, кто имеет на это право;
- целостность – информация сохраняется в полном объеме и не изменяется без ведома владельца;
- доступность – тот, кто имеет право на доступ к информации, может ее получить.

Большинство современных компаний ежедневно сталкиваются с задачей гарантировать бесперебойную работу своих сервисов. Сбои и простои могут привести к значительным финансовым потерям, урону репутации, а также к потере доверия клиентов и бизнес-партнеров. Именно поэтому создание надежной ИТ-инфраструктуры должно стать неотъемлемой частью стратегии любой компании, заботящейся о своем будущем.

Отказоустойчивость ИТ-инфраструктуры – это способность системы продолжать нормальную работу даже в случае частичных сбоев, неожиданных ошибок или выхода из строя отдельных элементов системы. Это означает, что если даже происходят сбои в отдельных частях ИТ-системы, то они не влекут за собой остановку всего сервиса в целом. В системе с высокой отказоустойчивостью подобные проблемы выявляются и устраняются автоматически, не требуя вмешательства администратора, и они не оказывают влияния на общую производительность или доступность системы [2].

В данной работе основное внимание будет уделено принципу доступности как важнейшему условию обеспечения отказоустойчивости современных информационных систем.

Объектом исследования являются микросервисные информационные приложения.

Предметом исследования, в свою очередь, является построение отказоустойчивой информационной системы с применением инструментов контейнеризации и оркестрации контейнеров.

Целью выпускной квалификационной работы является разработка отказоустойчивой информационной системы, основанной на микросервисной архитектуре, с применением лучших практик обеспечения отказоустойчивости.

Задачами выпускной квалификационной работы являются.

1. Анализ актуальности микросервисной архитектуры для построения отказоустойчивых информационных систем.
2. Сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем.
3. Разработка архитектуры отказоустойчивой микросервисной информационной системы.
4. Реализация прототипа отказоустойчивой микросервисной системы.
5. Тестирование отказоустойчивости прототипа.

ГЛАВА 1. АНАЛИЗ ПРИНЦИПОВ И МЕТОДОВ ПОСТРОЕНИЯ ОТКАЗОУСТОЙЧИВЫХ ИНФОРМАЦИОННЫХ СИСТЕМ

1.1 Анализ актуальности микросервисной архитектуры для построения отказоустойчивых информационных систем

Современные требования к надежности и непрерывности работы информационных систем особенно актуальны для объектов критической информационной инфраструктуры (КИИ), где сбой в работе может привести к значительным экономическим потерям, нарушению производственных процессов или угрозе безопасности. В таких условиях особое внимание уделяется внедрению архитектурных подходов, обеспечивающих отказоустойчивость и масштабируемость систем.

Микросервисная архитектура, благодаря своей модульности и независимости компонентов, позволяет минимизировать последствия сбоев отдельных сервисов, упрощает обновление и восстановление системы без остановки ее работы, что делает ее эффективным подходом для построения отказоустойчивых систем, в том числе для объектов критической информационной инфраструктуры.

Микросервисная архитектура формирует основу для развития крупных цифровых продуктов и экосистем, где важны масштабируемость, независимость компонентов и отказоустойчивость. Так, например, «Яндекс Go» приложение, которое объединяет службу заказа такси, доставку еды, маркет и другие сервисы. Все функции реализованы как отдельные микросервисы, которые могут развиваться независимо, масштабироваться под свою нагрузку и обновляться без остановки остального приложения. Это позволяет быстро добавлять новые направления, интегрировать внешние сервисы и эффективно работать с пиковыми нагрузками.

Или другой пример, компания «Netflix», которая в начале 2010-х столкнулась с тем, что классический монолит не справлялся с нагрузкой от миллионов пользователей по всему миру. Компания постепенно перешла к микросервисной архитектуре: каждое направление – от рекомендаций до

стриминга видео и выставления счетов – стало отдельным сервисом со своей командой. Это позволяет «Netflix» развиваться на новых рынках, запускать новинки и оставаться лидером отрасли по скорости внедрения изменений [3].

Приведенные примеры демонстрируют, что микросервисная архитектура позволяет строить отказоустойчивые, масштабируемые и гибкие системы, способные быстро адаптироваться к растущим нагрузкам и требованиям пользователей. Такой подход становится все более актуальным не только для крупных цифровых платформ, но и для объектов критической информационной инфраструктуры, где непрерывность работы систем имеет критическое значение.

1.2 Планирование информационной системы

Отказоустойчивая информационная система должна начинаться с планирования инфраструктуры и проработки архитектуры. Также хорошей практикой будет определение уровня обслуживания.

Соглашение об уровне обслуживания (Service-level agreement, SLA) – это соглашение между поставщиком услуг и заказчиком. Уровни обслуживания – однозначные и измеримые показатели, которые поставщик услуг обязуется соблюдать, например: процент доступности сервиса, время восстановления работы оборудования и сервисов или время ответа службы поддержки. SLA – это стандартный документ для телекоммуникационных и ИТ-компаний, который имеет большое значение для грамотного управления ожиданиями пользователей и создания прочных деловых отношений [4].

Один из показателей SLA, который может быть включен в соглашение это – доступность услуги. Этот показатель измеряет количество времени, в течение которого служба доступна для использования, в процентном соотношении. Услуга не может быть доступна на 100%, но должна стремиться к этому значению. Таблица 1.1 показывает время недоступности, допустимое для каждого уровня доступности. Рассчитать время доступности можно по формуле (1.1), а по формуле (1.2) – время недоступности.

Таблица 1.1 – Время недоступности, допустимое для каждого уровня доступности услуги

Доступность услуги, %	Время простоя в год	Время простоя в месяц (30 дней)
99% - «две девятки»	87.6 ч	7.2 ч
99.9% - «три девятки»	8.76 ч	43.2 мин
99.99% - «четыре девятки»	52.56 мин	4.32 мин
99.999% - «пять девяток»	5.26 мин	25.92 сек
99.9999% - «шесть девяток»	31.54 сек	2.59 сек

$$uptime = total_time * \frac{SLA}{100\%} \quad (1.1)$$

$$downtime = total_time - uptime \quad (1.2)$$

На основе принятых соглашений и должна проектироваться система. Нужно понимать, чем строже будут требования, тем дороже выйдет инфраструктура и выше требуемая квалификация специалистов, способных настроить и поддерживать соответствующую архитектуру системы. Также необходимо учитывать тот факт, что со временем система, как правило, обростает новым функционалом, что поспособствует обновлению требований SLA, поэтому важно, чтобы система имела потенциал для масштабирования.

1.3 Отказоустойчивость на инфраструктурном уровне

Основная цель при построении надежных систем – гарантировать непрерывность работы ИТ-инфраструктуры и минимизировать время простоя, что способствует стабильности бизнес-процессов [2].

Основу отказоустойчивости составляет избыточность – наличие запасных компонентов для всех элементов системы [5].

Отказоустойчивость на инфраструктурном уровне достигается за счет применения аппаратного резервирования, технологий зеркалирования оперативной памяти (memory mirroring mode, режим зеркалирования памяти) [6], RAID массивов различных уровней (Redundant Array of Independent Disks, избыточный массив независимых дисков) [7, 8, 9], резервного копирования данных [10, 11] и георезервирования [5, 12, 13]. Дополнительную устойчивость обеспечивают кластеры высокой доступности (high availability

cluster, HA-кластер), перераспределяющие нагрузку при отказе отдельных узлов и минимизирующие простои [14]. Совокупность этих механизмов позволяет снизить риск потери данных и обеспечить бесперебойную работу инфраструктуры даже при аппаратных сбоях.

1.4 Отказоустойчивость на архитектурном уровне

1.4.1 Основа архитектурной устойчивости

Продуманная, отказоустойчивая инфраструктура – это только часть работы. Следующий этап заключается в том, как спроектировать отказоустойчивое приложение и определить оптимальную среду его развертывания.

Современные приложения могут быть реализованы как в виде монолита, так и с использованием микросервисной архитектуры. Взаимодействие с приложением может осуществляться синхронно через REST или асинхронно через очереди сообщений, например, Kafka. Важную роль играет и выбор среды развертывания, например приложение может работать в виртуальных машинах или в контейнерах, а контейнеры в свою очередь могут управляться оркестраторами, такими как Kubernetes или OpenShift. Кроме того, особое внимание следует уделять масштабированию и отказоустойчивости баз данных, используя различные подходы и техники.

1.4.2 Монолит или микросервисная архитектура

При проектировании отказоустойчивых информационных систем важным решением становится выбор архитектуры приложения. Существуют две основные архитектуры: монолитная и микросервисная.

Монолит представляет собой единую систему, в которой все компоненты приложения тесно связаны и разрабатываются как одно целое. Такой подход упрощает начальную разработку, отладку и развертывание, но имеет существенные ограничения при масштабировании и обновлении. Сбой в одной части приложения может повлечь за собой отказ всей системы, а изменения требуют повторной сборки и тестирования всего приложения.

Микросервисная архитектура, напротив, разбивает систему на независимые сервисы, каждый из которых выполняет конкретную задачу и взаимодействует с другими через стандартизированные интерфейсы. Это позволяет изолировать сбои, масштабировать отдельные части приложения, применять разные технологии для разных сервисов. Однако микросервисы предъявляют более высокие требования к организации сети, мониторингу, оркестрации и согласованности данных.

С точки зрения отказоустойчивости, микросервисный подход обеспечивает большую гибкость и устойчивость, но требует продуманной архитектуры и инфраструктуры.

Для сравнения монолитной и микросервисной архитектуры удобно воспользоваться таблицей 1.2 [15].

Таблица 1.2 – Сравнение монолитной и микросервисной архитектуры

Характеристика	Монолитная архитектура	Микросервисная архитектура
Разработка и развертывание	одна база кода; простые процессы разработки и развертывания	децентрализованная разработка; каждый сервис разрабатывается и развертывается по отдельности
Масштабируемость и производительность	масштабируемость всего приложения; возможно нерациональное использование ресурсов	детализированная масштабируемость; оптимальное распределение ресурсов, исходя из потребностей различных сервисов
Обслуживание и расширяемость	единая база кода; проще понять и легче обслуживать	независимые сервисы; проще добавлять новые функции, не изменяя все приложение
Технологическое разнообразие и автономность	ограниченный стек технологий; привязка к определенному набору технологий	разнообразие технологий, доступных для каждого сервиса; можно выбрать, какие технологии подойдут лучше
Отказоустойчивость	единая точка отказа; отказы влияют на все приложение	изолированные отказы; отказоустойчивость лучше

Продолжение таблицы 1.2

Характеристика	Монолитная архитектура	Микросервисная архитектура
Координация и взаимодействие	прямые вызовы функций; более простая коммуникация и обмен данными	взаимодействие между сервисами; дополнительные средства для координации и согласованности данных

Схематично монолитная и микросервисная архитектура показана на рисунке 1.1. На данной схеме видны основные преимущества микросервисного подхода. Так, если нагрузка на сервис оплаты возрастет, в монолите придется поднимать второй экземпляр приложения, в котором будут сервисы, не нуждающиеся на данный момент в масштабировании. Отсюда и нерациональный подход к использованию ресурсов. В то же время микросервисная архитектура, позволяет увеличить количество экземпляров конкретного нагруженного сервиса.

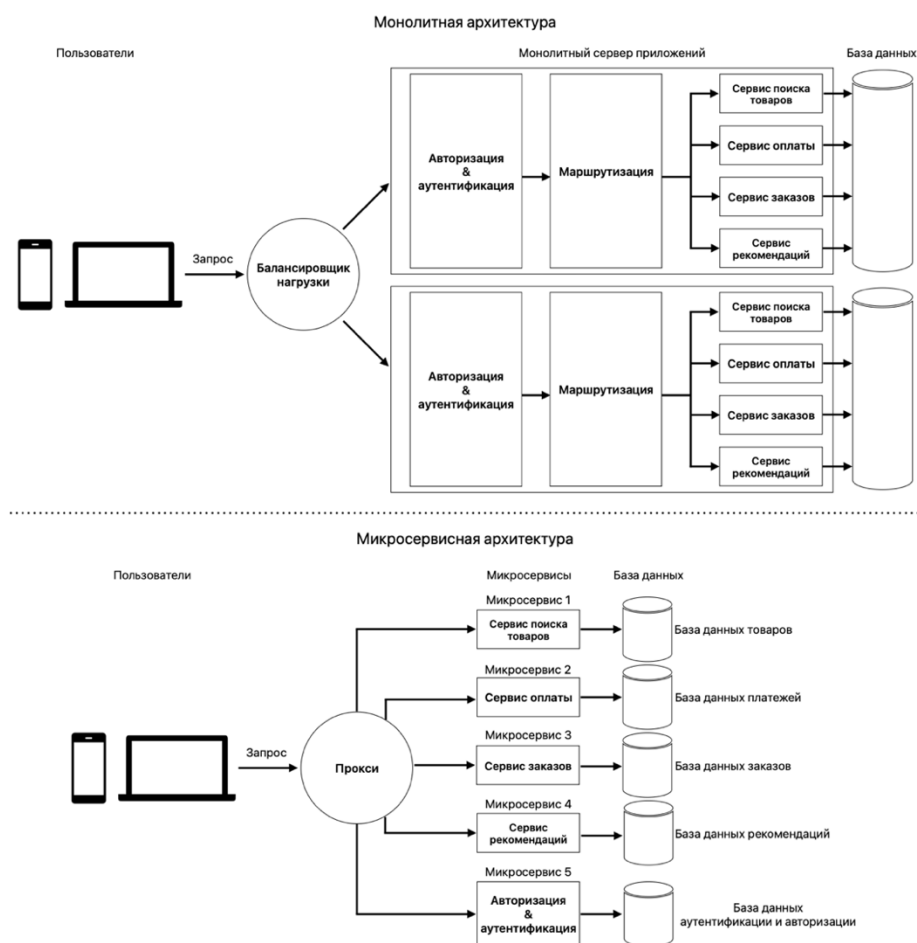


Рисунок 1.1 – Схема монолитной и микросервисной архитектуры

Также не стоит забывать и про другой сценарий, при котором существует только один экземпляр монолита и по одному экземпляру в микросервисах. В случае отказа сервиса оплаты, в монолите будет потеряна работоспособность всего приложения, в то время как при использовании микросервисной архитектуры, невозможно будет только совершить оплату, а все остальные функции приложения продолжат свою работу.

1.4.3 Виртуальные машины и контейнеры

Современные отказоустойчивые информационные системы все чаще разворачиваются в изолированных средах – виртуальных машинах (ВМ) или контейнерах. Обе технологии позволяют добиться изоляции процессов, гибкости при масштабировании, быстрого восстановления после сбоев и эффективного использования ресурсов.

Виртуальные машины предоставляют полную эмуляцию физического сервера, включая собственную операционную систему. Это делает их универсальными и независимыми от основной ОС хоста, но требует значительных ресурсов и времени на запуск. Виртуализация хорошо подходит для изоляции приложений.

Контейнеры, в отличие от виртуальных машин, используют ядро операционной системы хоста, что делает их легкими, быстрыми и идеальными для микросервисной архитектуры. Контейнеры запускаются практически мгновенно, занимают меньше ресурсов и лучше масштабируются. Их легко интегрировать с системами оркестрации – например, Kubernetes или OpenShift.

Для сравнения виртуальных машин и контейнеров удобно воспользоваться таблицей 1.3 [16].

Таблица 1.3 – Сравнение виртуальных машин и контейнеров

Характеристика	Виртуальная машина	Контейнер
Изоляция процесса	Изоляция процесса на аппаратном уровне	Изоляция процесса на уровне ОС
Операционная система	Каждая виртуальная машина имеет отдельную ОС	Каждый контейнер может совместно использовать ОС

Продолжение таблицы 1.3

Характеристика	Виртуальная машина	Контейнер
Время загрузки	Загружается в считанные минуты	Загружается в считанные секунды
Занимаемое место	Виртуальные машины занимают несколько ГБ	Контейнеры легкие (КБ / МБ)
Доступность готовых решений	Готовые виртуальные машины трудно найти	Готовые контейнеры легко доступны
Перемещение на новый хост	Виртуальные машины могут перемещаться на новый хост	Контейнеры уничтожаются и воссоздаются, а не перемещаются
Время создания	Создание виртуальной машины занимает относительно больше времени	Контейнеры могут быть созданы в считанные секунды
Использование ресурсов	Требуют больше ресурсов	Требуют меньше ресурсов

Схема, сравнивающая виртуальные машины и контейнеры, показана на рисунке 1.2.

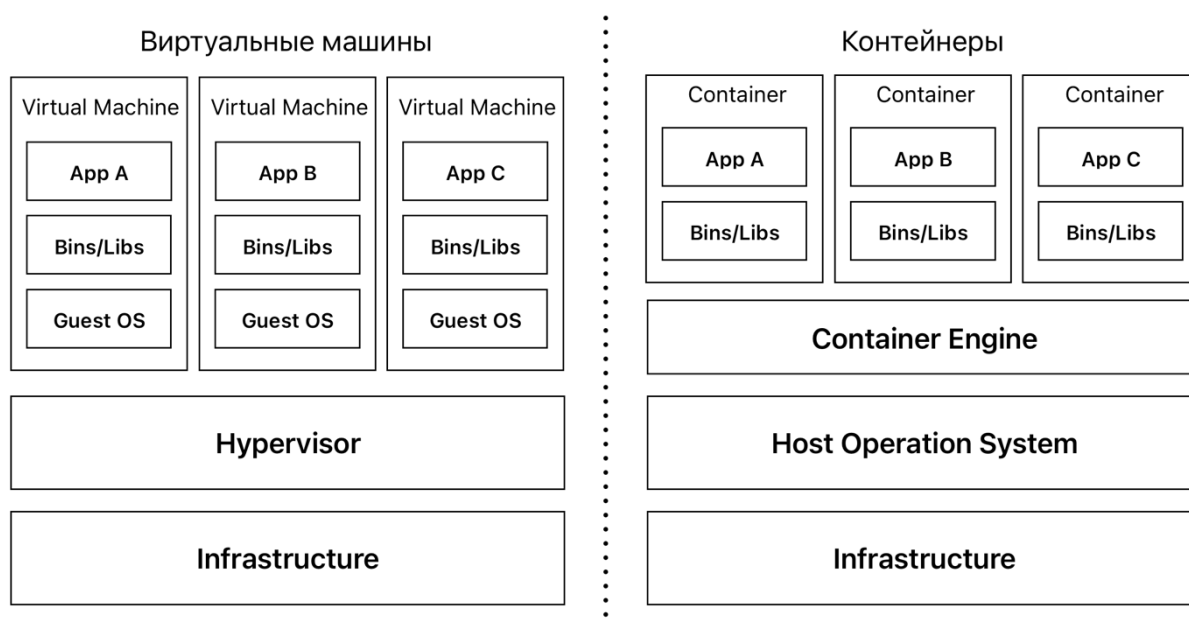


Рисунок 1.2 – Схема виртуальных машин и контейнеров

Провести границу, когда и какую технологию разворачиванию лучше использовать довольно трудно. Часто эти технологии используются вместе. Но тем не менее реализовывать отказоустойчивую информационную систему на основе микросервисов значительно удобнее и эффективнее с помощью контейнеров.

1.4.4 Оркестраторы контейнеров

С развитием контейнеризации появилась необходимость в управлении большим числом контейнеров, их взаимодействием, масштабированием и отказоустойчивостью. Решением этой задачи стали оркестраторы контейнеров – инструменты, которые автоматизируют развертывание, управление, масштабирование и восстановление контейнеризированных приложений.

Одним из наиболее популярных оркестраторов является Kubernetes (K8s), который предоставляет полный набор механизмов для построения отказоустойчивых систем.

Основные преимущества Kubernetes.

- Автоматическое восстановление. Kubernetes автоматически перезапускает поды, которые перестали работать, перемещает поды на рабочие узлы, убирает и заменяет поды, не прошедшие проверку состояния.

- Масштабируемость. K8s позволяет легко масштабировать приложения как вертикально (добавляя больше ресурсов для каждого отдельного контейнера), так и горизонтально (добавляя больше экземпляров подов).

- Автоматизация развертывания и управления. Kubernetes предоставляет ряд API и механику работы отдельных ресурсов, которые позволяют построить инструменты для автоматического развертывания, обновления и отката приложений. С их помощью можно интегрировать процессы CI/CD и обеспечить быстрое и надежное обновление приложений без простоев.

- Безопасность. Kubernetes обеспечивает изоляцию приложений на уровне контейнеров, а также изоляцию от слоя управления кластером.

- Балансировка нагрузки. Kubernetes автоматически распределяет трафик между подами, обеспечивая высокую доступность и оптимальную производительность приложения.

- Экономия ресурсов. K8s управляет легковесными контейнерами. Это позволяет эффективно использовать вычислительные ресурсы и снижает затраты на инфраструктуру.

– Поддержка микросервисной архитектуры. Kubernetes позволяет управлять множеством небольших, независимо разрабатываемых и развертываемых сервисов. Это позволяет эффективно масштабировать систему, стандартизировать подходы, упрощать обновление кода и продуктов, а также обеспечивать легкость отката изменений [17].

Недостатком Kubernetes является высокий порог вхождения. Платформа представляет собой сложную систему с большим количеством абстракций, что требует значительных временных затрат на обучение и освоение технологии.

Оркестраторы контейнеров являются неотъемлемой частью современной архитектуры отказоустойчивых систем, обеспечивая высокий уровень автоматизации и наблюдаемости (observability). Они обеспечивают приложениям устойчивость к сбоям и изменяющимся нагрузкам, а также поддерживают самовосстановление и непрерывную работу в любых условиях.

1.4.5 Масштабирование базы данных

База данных (БД), как и любой компонент системы, должна масштабироваться для повышения отказоустойчивости. Горизонтальное масштабирование (добавление новых серверов или узлов) для БД по умолчанию недоступно, так как это может привести к нарушению консистентности данных. Но существуют различные техники масштабирования, которые открывают такую возможность, например репликация или шардирование.

Техника репликации заключается в создании и поддержании нескольких копий данных на разных серверах или узлах. Эта техника обеспечивает высокую доступность и отказоустойчивость базы данных. В типичной модели репликации «primary-replica» один узел назначается primary (лидер), а остальные становятся replica (копиями):

– лидер обрабатывает все операции записи, обеспечивая их согласованность и целостность. При изменении или добавлении данных в базу лидера, эти изменения автоматически распространяются на узлы-реплики;

— лидер также может обрабатывать критические операции чтения, где требуется высокая степень согласованности. Реплики обычно используются для обработки запросов на чтение, чтобы распределить нагрузку и улучшить производительность системы;

— если лидер выйдет из строя, то одна из реплик станет лидером.

Схема репликации показана на рисунке 1.3. Преимуществами данного подхода является улучшенная производительность на чтение и высокая доступность, даже в случае сбоя на некоторых узлах. А недостатками данного подхода является задержка в синхронизации данных и сложность реализации [18].

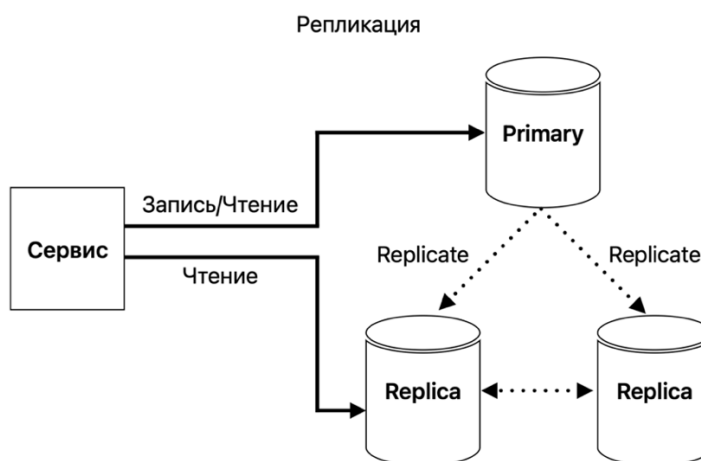


Рисунок 1.3 – Схема репликации базы данных

При использовании техники шардирования одна большая база данных разделяется на более мелкие и управляемые БД, которые называются шардами. Основные стратегии разделения базы:

— шардирование по диапазону значений — данные распределяются между шардами по диапазонам ключей (например, ID от 1 до 1000 — в одном шарде, от 1001 до 2000 — в другом);

— хеш-шардирование — к значению ключа применяется хеш-функция, результат которой определяет, в каком шарде будут храниться данные;

— директорное шардирование — используется отдельная таблица-сопоставление, которая указывает, где именно находится каждая запись.

Схема шардирования на основе диапазона ключей показана на рисунке 1.4. Преимущество шардирования заключается в значительном повышении производительности при запросах и операциях записи, так как они могут обрабатываться параллельно, если направлены в разные шарды. К недостаткам относят перебалансировку данных между шардами, которая может быть сложной и времязатратной процедурой, и объединение данных между шардами, которая может стать нетривиальной задачей [18].

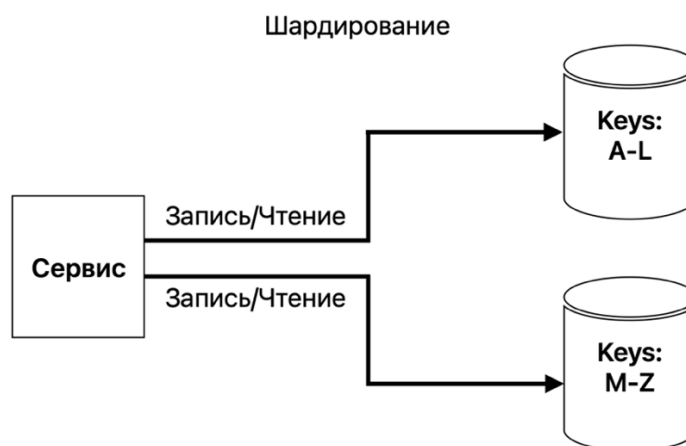


Рисунок 1.4 – Схема шардирования базы данных на основе диапазона ключей

1.4.6 Балансировка нагрузки

При масштабировании информационных систем по горизонтали – будь то отдельные серверы или микросервисы – критически важно обеспечить корректное распределение входящего трафика. Без этого новые ресурсы не будут эффективно использоваться. Решением этой задачи занимаются сервисы балансировки нагрузки, например, Nginx.

Балансировка нагрузки – это распределение входящего сетевого трафика по нескольким серверам или сервисам. Распределение нагрузки обеспечивает стабильную, масштабируемую и отказоустойчивую работу системы, исключая ситуации перегрузки отдельных компонентов.

Балансировщик выступает в качестве промежуточного звена между клиентами и серверами, его задача состоит в том, чтобы принимать входящие запросы от клиентов и перенаправлять их на один из доступных серверов в зависимости от выбранной стратегии распределения нагрузки. Основная цель

– обеспечить, чтобы ни один сервер не был перегружен, так это может привести к снижению производительности или даже к отказу в обслуживании.

Среди стратегий балансировки нагрузки распространены следующие:

– Round-robin – запросы поочередно распределяются между всеми доступными серверами.

– Least-connections – запросы отправляются на сервер с наименьшей текущей нагрузкой (минимальное количество открытых соединений).

– IP-hash – балансировщик нагрузки вычисляет хэш от IP-адреса клиента и на основании этого хэша определяет, на какой сервер направить запрос.

– URL-hash – подход похожий на IP-hash, только хэш вычисляется на основе URL-запроса.

Применение балансировки нагрузки не только распределяет нагрузку и улучшает общую производительность, но также повышает общую отказоустойчивость системы. В случае сбоя одного из серверов, балансировщик автоматически исключает его из пула и перенаправляет трафик на оставшиеся узлы, обеспечивая непрерывность обслуживания пользователей. Таким образом, балансировка нагрузки является неотъемлемой частью архитектуры отказоустойчивых информационных систем.

1.5 Анализ лучших практик построения отказоустойчивых информационных систем

1.5.1 DevOps

Основной подход в DevOps – это автоматизация разработки, тестирования и развертки приложений. Благодаря ей, команда может ускорить и стандартизировать все процессы и создать цикл непрерывной интеграции (Continuous Integration) и непрерывного развертывания (Continuous Deployment) приложений – CI/CD-конвейер [19].

Автоматизация позволяет уменьшить ошибки из-за человеческого фактора до минимума. К тому же, благодаря грамотно настроенному CI/CD

конвейеру можно увеличить отказоустойчивость приложения за счет быстрой доставки кода на различные среды тестирования.

1.5.2 Continuous Integration и Continuous Delivery / Deployment

CI/CD-конвейер предназначен для автоматизации всех этапов разработки и развертывания, что позволяет сокращать время выхода новых версий, снижать вероятность ошибок и повышать стабильность и предсказуемость поставки программного продукта.

Если разделить CI/CD на составляющие, то получится следующие:

– CI (Continuous Integration) – непрерывная интеграция, представляет собой процесс, где написанный код автоматически собирается и проходит unit тестирование, это позволяет выявить возможные ошибки на ранних стадиях;

– CD (Continuous Delivery или Deployment) – непрерывная доставка или развертывание, представляет собой доставку собранного приложения на различные стенды, включая продакшен среду. Отличие delivery от deployment заключается в том, что в delivery релизы новых версий продукта начинается при нажатии на кнопку, в то время как в deployment релиз проходит автоматически.

Схема CI/CD-конвейера показана на рисунке 1.5. Данные конвейер, это не единственно верная реализация. Для каждого проекта конвейер может быть свой, он может включать в себя дополнительные этапы, как в части CI, так и в части CD. Выбор между delivery и deployment, также лучше делать с учетом специфики проекта.

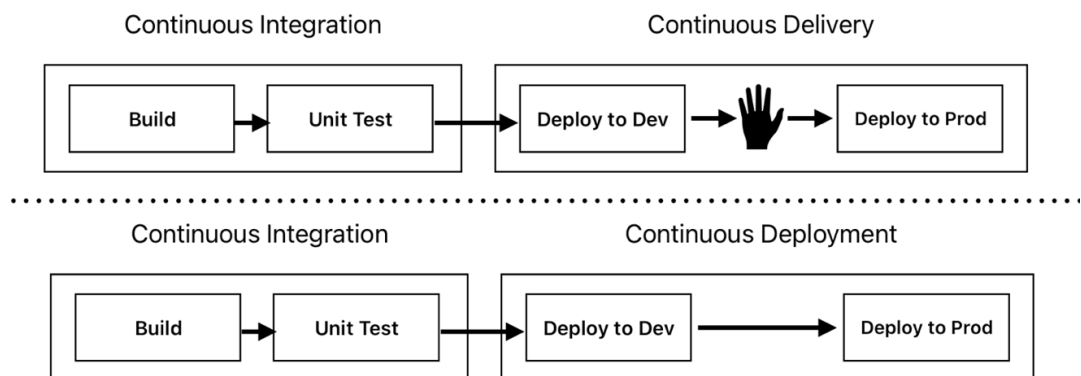


Рисунок 1.5 – Схема CI/CD конвейера

Пример CI части из реального проекта показан на рисунке 1.6. Данный конвейер реализован в Jenkins. Помимо сборки и тестирования проекта, здесь применяются практики DevSecOps, включающие сканирование кодовой базы на наличие уязвимостей с использованием SAST (Статический анализ безопасности приложений, Static Application Security Testing) и SCA (Анализ состава программного обеспечения, Software Composition Analysis), собирается Docker образ и публикуется в registry.

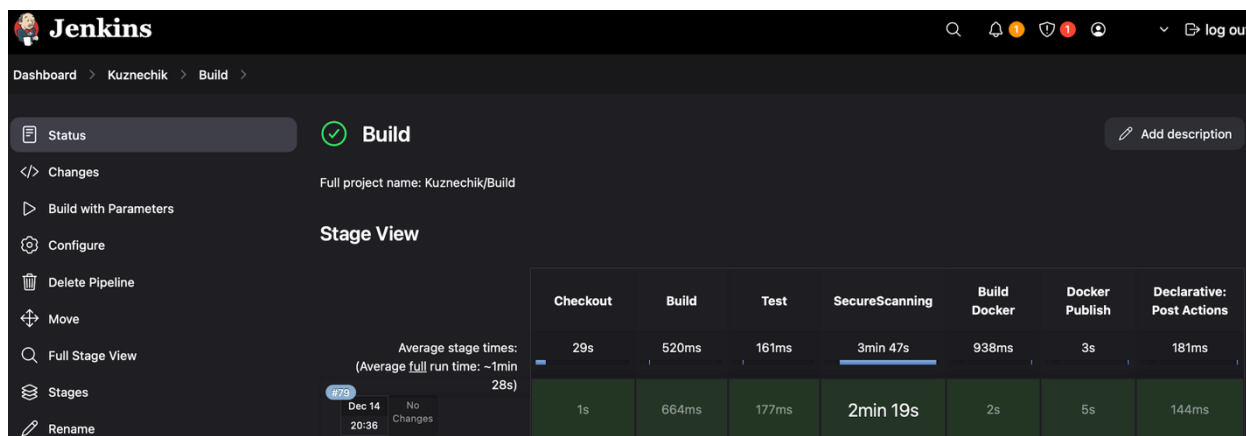


Рисунок 1.6 – CI-конвейер в Jenkins

1.5.3 Среды развертывания

Для повышения отказоустойчивости системы приложение должно быть тщательно протестировано. Для этого выделяют различные среды развертывания. CI/CD-конвейер значительно ускоряет и упрощает развертывание различных версий приложений, однако каждую из сред необходимо поддерживать и настраивать, что требует дополнительных затрат.

Чаще всего распространены следующие среды развертывания:

- Dev (Development) – среда разработки, как правило, здесь проходят первые этапы тестирования с замканными сервисами;
- Test (Testing) – среда тестирования, здесь проверяется корректность интеграционного взаимодействия с другими сервисами;
- Stage (Staging) – среда максимально приближенная к продакшену, на которой проходит финальный этап тестирования приложения;

– Prod (Production) – продакшен среда, на которой работают пользователи приложения.

Сред развертывания может быть как больше, так и меньше и использоваться они могут с разной целью.

1.5.4 Стратегии развертывания приложений

Как бы хорошо приложение не было протестировано, всегда есть вероятность, что при релизе новой версии продукта, произойдет сбой и приложение упадет с ошибкой. Также бывают ситуации необходимости проверки продуктовых гипотез. Для того чтобы минимизировать вероятность отказа при деплое новой версии приложения или реализовать проверку гипотезы, существуют различные стратегии развертывания приложений.

Стратегии деплоя или развертывания определяют, как будут запускаться приложения/сервисы [20]. В проекте может использоваться как одна стратегия, так и применяться сразу несколько. Распространены следующие стратегии: Recreate, Rolling, Blue/Green, Canary, A/B-тестирование.

Recreate (Повторное создание). Стратегия состоит из двух шагов: удаление текущей версии приложения и развертывание новой. Это самая простая стратегия. Если возникнет какая-то ошибка, то придется откатиться к предыдущей версии, а это бывает не всегда просто сделать. Стратегия развертывания Recreate показана на рисунке 1.7.

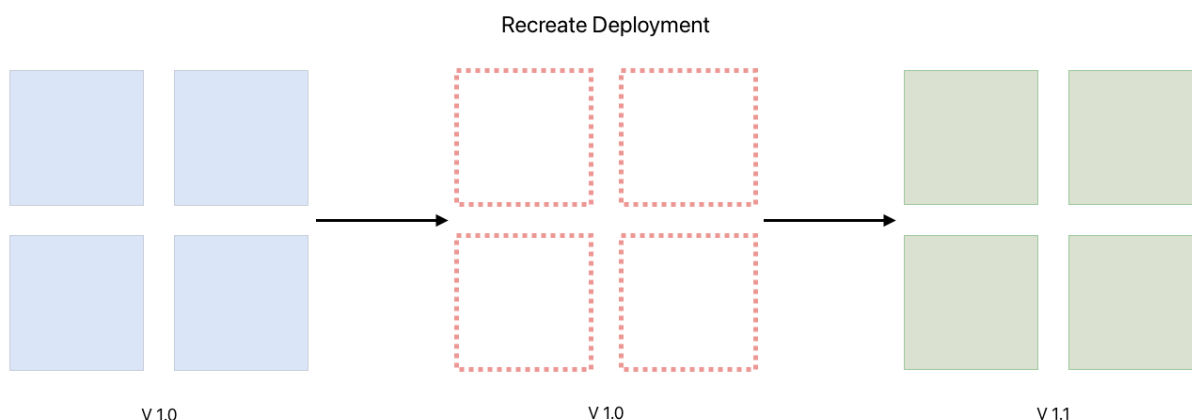


Рисунок 1.7 – Стратегия развертывания Recreate

Rolling (Постепенное развертывание). В этой стратегии все экземпляры приложения будут последовательно обновляться до новой версии. Каждый экземпляр проходит следующий цикл: удаляется один из экземпляров со старой версией приложения (остальные продолжают работать), далее запускается экземпляр с новой версией, затем проверяется корректный запуск нового экземпляра и если все хорошо, то цикл повторяется для следующего экземпляра. Стратегия развертывания Rolling показана на рисунке 1.8.

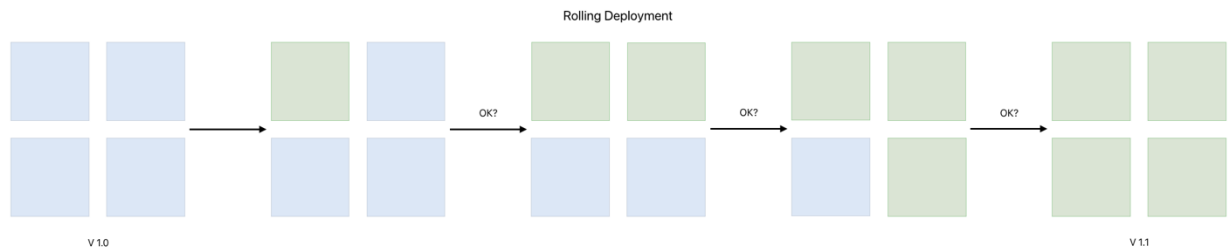


Рисунок 1.8 – Стратегия развертывания Rolling

Blue/Green (Сине-зеленое развертывание). Эта стратегия базируется на двух продуктивных средах: синей и зеленой. В синей среде находится старая версия приложения, а в зеленой – будет запущена новая версия. Когда зеленая среда после тщательного тестирования готова принять на себя пользователей, на нее переключается пользовательский трафик. Если с новой версией в зеленой среде возникают проблемы, то трафик возвращается на старую версию приложения, то есть в синюю среду. Стратегия развертывания Blue/Green показана на рисунке 1.9.

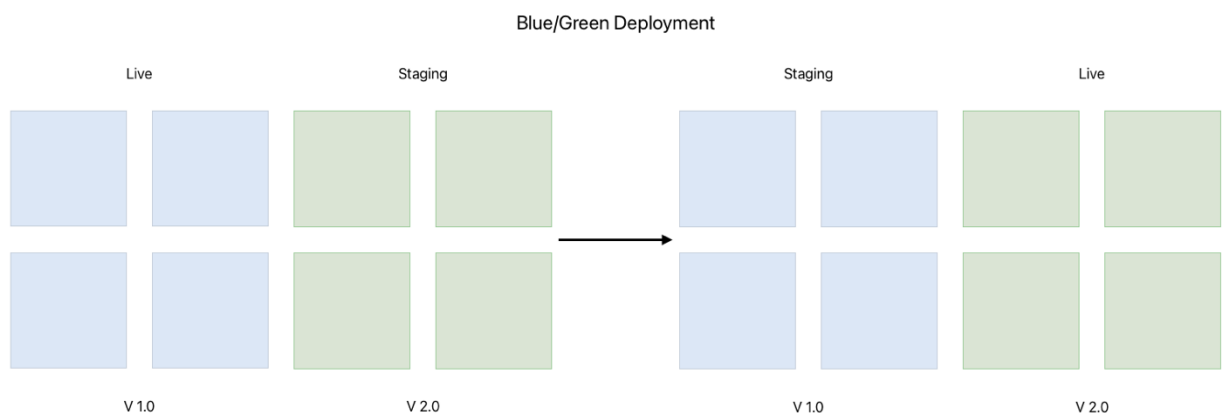


Рисунок 1.9 – Стратегия развертывания Blue/Green

Canary (Канареечное развертывание). Канареечный деплой схож с синей-зеленой стратегией. Отличие в том, что пользователи приложения постепенно переключаются на новую версию. Часть текущих экземпляров приложения заменяется новой версией, на которую переключается часть трафика. Если новая версия работает стабильно, то доля пользователей, использующих новую версию приложения, постепенно увеличивается. Так происходит до тех пор, пока все неактуальные экземпляры не будут заменены. Если же в процессе обновления возникают проблемы, то трафик можно быстро вернуть на предыдущую версию. Стратегия развертывания Canary показана на рисунке 1.10.

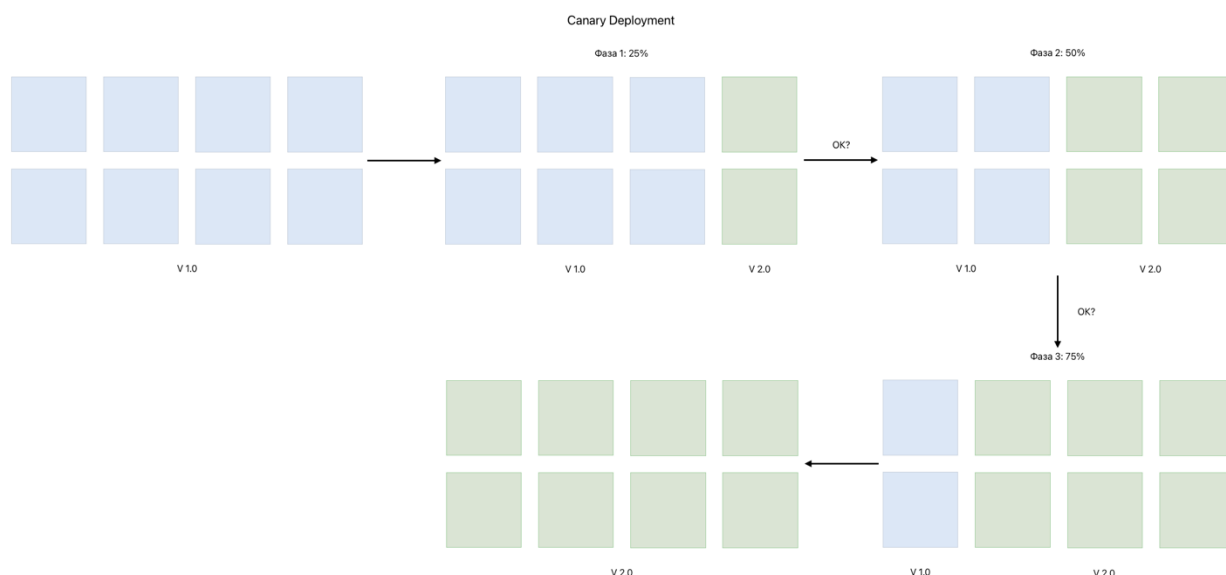


Рисунок 1.10 – Стратегия развертывания Canary

А/В-тестирование. Фактически это вариация канареечного развертывания применительно для фронтенда. Здесь также запускается ограниченное количество экземпляров с новой версией приложения, на которую переключается часть пользовательского трафика. Эта стратегия используется больше в экспериментах, для проверки идей и гипотез. Например, с помощью такой стратегии релиза можно узнать, как пользователи реагируют на те или иные изменения. Стратегия А/В-тестирования при развертывании показана на рисунке 1.11.

Восстановление после сбоев (Disaster Recovery, DR) – это стратегия, которая включает в себя процедуры и политики для восстановления критических функций ИТ-системы после возникновения сбоя или катастрофы [2].

Цель DR – минимизировать время простоя и предотвратить потерю данных. Весь процесс DR подразделяется на четыре этапа:

- оценка рисков и воздействия;
- разработка DR-плана;
- тестирование плана;
- реализация плана после возникновения сбоя.

В плане восстановления после сбоя описываются действия, которые нужно предпринять, а также объясняется, как эти действия будут поддерживать бизнес-процессы. Важно понимать, что не существует универсального идеального плана. План уникален для каждой системы, так как он учитывает всю ее специфику и особенности.

1.6 Сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем

1.6.1 Контейнерные платформы

Docker и Podman – одни из популярных платформ контейнеризации. Поскольку Podman и Docker предназначены для решения одних и тех же задач, между ними много общего, но также есть и несколько принципиальных различий.

С точки зрения архитектуры, Docker использует клиент-серверную логику и демон (постоянно работающую фоновую программу) для создания образов и запуска контейнеров, в то время как в архитектуре Podman демон не используется, поэтому он может запускать контейнеры от имени пользователя. Для управления сервисами и поддержки запуска контейнеров Podman в фоновом режиме применяет systemd, который создает контрольные юниты для существующих контейнеров или для генерации новых.

С точки зрения безопасности настройки Podman по умолчанию включают запуск контейнеров без root привилегий в изолированных пользовательских пространствах имен, что ограничивает последствия потенциального выхода из контейнера. В противоположность этому, настройки Docker по умолчанию запускают процессы контейнера от имени пользователя root, что представляет более высокий риск в случае выхода из контейнера. Однако Docker также можно настроить для обеспечения максимальной безопасности, при этом Podman обычно требует меньше усилий для достижения безопасной конфигурации.

Экосистема Docker опережает Podman по масштабу и зрелости. Так Docker Hub служит стандартом для предварительно собранных образов контейнеров и содержит огромное количество репозиторий с миллионами образов для различных приложений, библиотек и фреймворков. Помимо этого, Docker обладает преимуществом и в поддержке сообщества, что может существенно повлиять на принятие решения о выборе инструмента.

Выбор между Docker и Podman зависит от конкретных требований. Docker предлагает надежную платформу с развитой экосистемой и широким распространением, что делает его отличным выбором для многих проектов. Podman же представляет собой легковесную альтернативу без демона, ориентированную на безопасность и удобство работы в средах с ограниченными привилегиями, что может быть более подходящим для определенных сценариев использования [21].

1.6.2 Оркестраторы контейнеров

Kubernetes не единственное решение для оркестрации контейнеров. Не менее популярным инструментом является OpenShift, который построен на базе Kubernetes.

Kubernetes – проект с открытым исходным кодом, в то время как OpenShift представляет собой коммерческое корпоративное решение.

С точки зрения развертывания Kubernetes требует ручной настройки и конфигурирования, тогда как OpenShift предлагает автоматизированную установку и настройку с интегрированными инструментами, упрощающие конфигурирование и управление кластером.

Со стороны безопасности Kubernetes предлагает несколько методов аутентификации и авторизации, которые разработчик может настраивать вручную, обеспечивая гибкость для создания индивидуальных решений. В то же время OpenShift предлагает строгие политики безопасности и встроенные функции защиты, удовлетворяющие большинству требований корпоративной безопасности.

Обновления в Kubernetes выполняются без простоев вручную с помощью командной строки `kubectl`. OpenShift автоматизирует процесс управления обновлениями и исправлениями, гарантируя, что кластеры и приложения остаются актуальными с минимальным ручным вмешательством. Это снижает риск простоя во время обновлений и гарантирует, что платформа использует новейшие функции безопасности.

Поддержка Kubernetes осуществляется силами большого сообщества, доступно множество ресурсов, включая форумы, репозитории GitHub и онлайн-руководства. OpenShift поставляется с корпоративной поддержкой от Red Hat. Она включает в себя доступ к обновлениям, исправлениям и профессиональную круглосуточную помощь, а также инициативный мониторинг.

Таким образом, Kubernetes обеспечивает гибкость и расширяемость, что подходит для организаций, обладающих опытом управления и настройки среды оркестрации контейнеров. Открытый исходный код и обширная экосистема предоставляют возможности для инноваций, однако эта гибкость имеет свою цену, поскольку требует больше ручной настройки и управления. В свою очередь OpenShift – интегрированное и готовое к корпоративному использованию решение со встроенными инструментами и поддержкой,

подходит для организаций, которым требуется оптимизированный процесс развертывания и надежная защита [22].

1.7 Разработка архитектуры отказоустойчивой микросервисной информационной системы

Разработка архитектуры отказоустойчивой микросервисной информационной системы начинается с проектирования структуры сервисов и их взаимодействия. Важно разделить систему на независимые сервисы, каждый из которых отвечает за конкретную бизнес-функцию, и определить границы их ответственности. Такая декомпозиция позволяет изолировать сбои, минимизировать влияние отказов одного сервиса на работу всей системы и облегчить масштабирование отдельных компонентов.

Особое внимание уделяется схемам коммуникации между сервисами. Для синхронного взаимодействия используются REST API, а для асинхронного – очереди сообщений, например Kafka. В архитектуре также учитываются критические точки отказа и проектируются механизмы повторных попыток (retry), таймаутов (timeout), автоматического выключателя (circuit breaker) и резервного варианта (fallback), что повышает устойчивость системы к сбоям.

При проектировании учитывается среда развертывания и инструменты оркестрации контейнеров (например, Kubernetes или OpenShift), что позволяет реализовать автоматическое масштабирование, балансировку нагрузки и самовосстановление сервисов. Дополнительно внимание уделяется масштабированию и отказоустойчивости баз данных, применению репликации и шардирования, что обеспечивает непрерывность работы всей системы даже при сбоях отдельных компонентов.

1.8 Постановка задачи

Целью выпускной квалификационной работы является разработка отказоустойчивой информационной системы, основанной на микросервисной

архитектуре, с применением лучших практик обеспечения отказоустойчивости.

Разрабатываемая система – это приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик») на C++ и взаимодействием по REST API на Python [23]. Эффективность решения будет проверена с помощью тестирования.

Для разработки отказоустойчивой информационной системы необходимо решить следующие задачи:

1. спроектировать архитектуру приложения;
2. реализовать блочный алгоритм шифрования ГОСТ Р 34.12–2015 («Кузнечик») на C++;
3. реализовать веб-сервисы с использованием FastAPI на Python;
4. реализовать Docker образы для каждого микросервиса;
5. реализовать CI-конвейер для автоматизации сборки и публикации Docker образов микросервисов;
6. реализовать Kubernetes манифесты;
7. развернуть приложение в Kubernetes;
8. провести тестирование разработанной информационной системы.

Вывод по 1 главе

В данной главе выпускной квалификационной работы проведен анализ актуальности микросервисной архитектуры для построения отказоустойчивых информационных систем, проанализированы принципы и методы построения отказоустойчивых информационных систем на инфраструктурном и архитектурном уровнях, проанализированы лучшие практики, проведен сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем, а также была проанализирована разработка архитектуры отказоустойчивой микросервисной информационной системы.

При проектировании отказоустойчивой системы важно понимать, что с повышением уровня отказоустойчивости возрастает и стоимость реализации.

Так дублирование компонентов, как правило, приводит к росту расходов без пропорционального повышения производительности. В то же время на архитектурном уровне, чем сложнее и надежнее система, тем выше требуемая квалификация сотрудников, способных эту систему настроить и поддерживать. Поэтому важно, чтобы затраты на систему были оправданы. Необходимо найти баланс между уровнем отказоустойчивости и реальными потребностями и возможностями бизнеса.

ГЛАВА 2. РЕАЛИЗАЦИЯ ПРОТОТИПА ОТКАЗОУСТОЙЧИВОЙ МИКРОСЕРВИСНОЙ СИСТЕМЫ

2.1 Структура проекта

Проект состоит из двух частей: фронтенда и бэкенда. В каждой из этих частей есть свои микросервисы, так в части фронтенда есть микросервис с UI, позволяющий взаимодействовать с бэкендом, а в бэкенде есть два микросервиса, реализующих функционал приложения.

Для каждого микросервиса реализован свой Dockerfile, который упрощает развертывание приложения. Также для микросервисов написаны Kubernetes манифесты Deployment и Service, которые необходимы для развертывания приложения в среде k8s.

Сборка образов проходит в Jenkins. Для этого реализован CI-пайплайн, который собирает и публикует образы контейнеров в удаленный реестр образов (registry). Также один из этапов конвейера включает в себя проверку кода на уязвимости, что способствует своевременному их выявлению и устранению.

Для управления исходным кодом всех компонентов проекта используется система Git, позволяющая отслеживать изменения и восстанавливать любые версии кода на разных этапах разработки.

2.2 Микросервисы

2.2.1 Фронтенд: UI

Данный микросервис называется «frontend». Фронтенд реализован в HTML, SCSS, JavaScript. При написании HTML-страницы использовалась методология БЭМ, стили страницы описаны в SCSS, который в дальнейшем компилируется в CSS. Использование препроцессора SCSS значительно упрощает работу со стилями и адаптацию страницы под различные версии браузеров, благодаря автоматическому добавлению вендорных префиксов. JS применяется для взаимодействия с бэкендом, чтобы передать и забрать результат работы сервиса.

Эти статические ресурсы фронтенда отдает Nginx сервер. Страница, с которой взаимодействует пользователь, показана на рисунке 2.1.

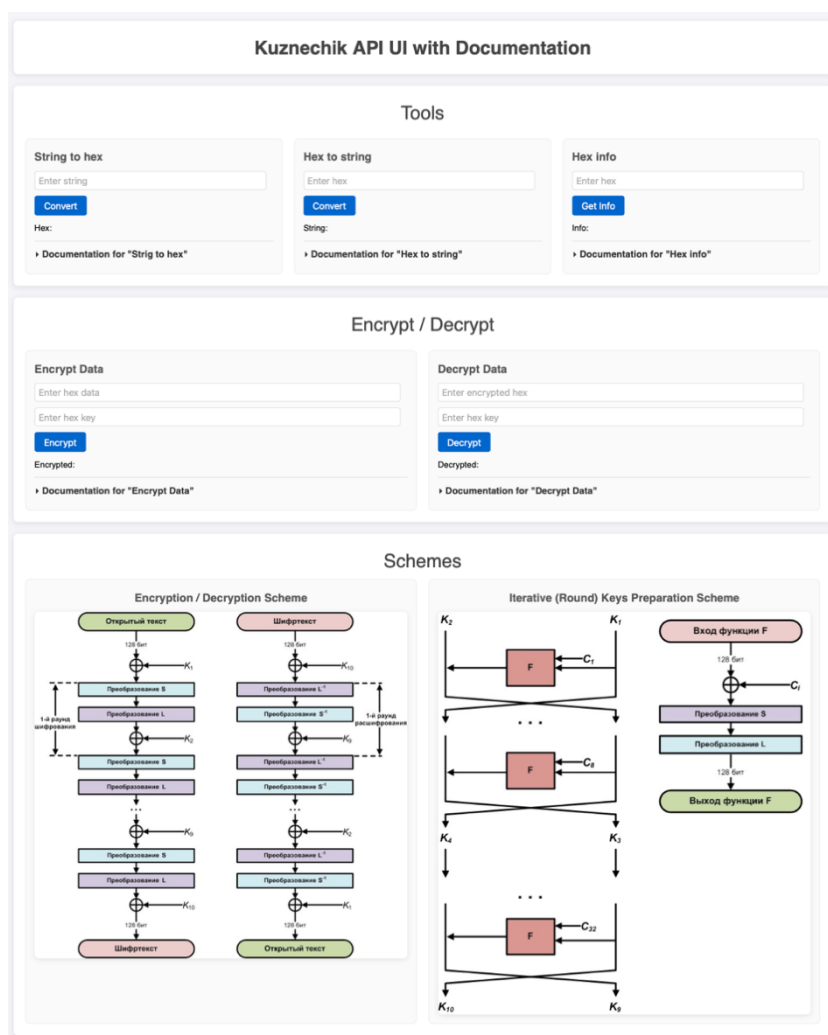


Рисунок 2.1 – UI часть проекта

2.2.2 Бэкенд: шифрование и дешифрование ГОСТ Р 34.12-2015

Данный микросервис называется «kuznechik_crypto». Микросервис написан на двух языках C++ и Python.

На C++ реализовано шифрование и дешифрование блока данных по алгоритму ГОСТ Р 34.12-2015. Код компилируется в shared library для интеграции взаимодействия с Python.

На Python реализовано API, при помощи веб-фреймворка FastAPI, которое взаимодействует с shared library. Помимо этого, на Python реализована дополнительная конечная точка /health/liveness, используемая Kubernetes для

проверки состояния сервиса. Также валидация входных параметров осуществляется на стороне Python.

Перечень всех конечных точек микросервиса «kuznechik_crypto»:

- /health/liveness;
- /kuznechik/encrypt;
- /kuznechik/decrypt.

2.2.3 Бэкенд: вспомогательные инструменты

Данный микросервис называется «tools». Он написан на Python и тоже использует веб-фреймворк FastAPI. Сервис предоставляет инструменты кодирования и декодирования строк в hex формат, а также дополнительную информацию по hex данным. Здесь также имеется валидация входных параметров и аналогичная конечная точка /health/liveness для Kubernetes.

Перечень всех конечных точек микросервиса «tools»:

- /health/liveness;
- /tools/str-to-hex;
- /tools/hex-to-str;
- /tools/hex-info.

2.3 Docker образы

2.3.1 Микросервис фронтенда

Docker образ данного микросервиса основан на Alpine с Nginx. Для сборки данного образа необходимы файлы фронтенда и конфигурационные файлы nginx сервера.

Конфигурация Nginx реализована модульным образом. Основной конфигурационный файл nginx.conf подключает конфигурацию frontend.conf, которая содержит конфигурацию веб-сервера с настройками порта и имени сервера, а также определяет базовый маршрут для обслуживания статического контента. Кроме того, через директиву include подключаются отдельные файлы с настройками проксирования HTTP-запросов к микросервисам. Каждый сервис представлен отдельным конфигурационным файлом,

расположенным в директории `proхu`. Такой подход упрощает сопровождение и обеспечивает масштабируемость системы.

2.3.2 Микросервис шифрования и дешифрования

Образ данного микросервиса собирается при помощи многоэтапной сборки (`multi-stage build`).

Для первого этапа используется базовый образ `Alpine`. На этом этапе выполняется подготовка и компиляции `C++` кода в `shared library`.

Для второго этапа используется образ `Alpine python` версии 3.10. На этом этапе выполняется подготовка `python` приложения, а также копируется собранная на предыдущем этапе библиотека.

Использование многоэтапной сборки позволяет не хранить зависимости, которые не будут использоваться в работе приложения, что уменьшает конечный вес образа, а также повышает безопасность системы путем минимизации окружения.

2.3.3 Микросервис инструментов

`Docker` файл для данного микросервиса похож на `Docker` файл сервиса шифрования и дешифрования, за тем исключением, что здесь не требуется предварительно компилировать `C++` код. Поэтому данный `Docker` образ собирается в один этап и использует в качестве базового такой же образ `Alpine python3.10`.

2.3.4 Использование хеш-сумм и минимальных базовых образов

Во всех `Docker` файлах базовые образы указываются с использованием их `sha256`-сумм. Такой подход обеспечивает воспроизводимость сборки, так как точно фиксирует используемую версию образа, в отличие от тегов, так, например, `тег latest` может указывать на разные версии с течением времени. Использование хеш-сумм также повышает безопасность: `тег` может быть переопределен и указывать на другой, потенциально недоверенный образ, тогда как `sha256` однозначно идентифицирует конкретный артефакт.

Также все образы строятся на основе легковесного дистрибутива Linux – Alpine. Для того чтобы узнать конечный вес получившихся образов микросервисов, можно собрать их локально с помощью команды, продемонстрированной в листинге 2.1, а затем посмотреть на их вес командой из листинга 2.2.

Листинг 2.1 – Сборка Docker образа

```
$ docker build -f <path_to_dockerfile> -t <microservice>:<version> .
```

Листинг 2.2 – Список локальных образов

```
$ docker images
```

Собранные локально образы показаны на рисунке 2.2. Вес каждого образа не превышает 100 мегабайт. Небольшой вес образов снижает сетевую нагрузку – это особенно важно при ограниченной пропускной способности, а также при частых обновлениях. Также это снижает затраты на хранение артефактов.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
kuznechik_crypto	0.0.1	8b58119b091f	6 minutes ago	84MB
tools	0.0.1	fe09a74c0e50	51 minutes ago	78.4MB
frontend	0.0.1	2b341ddfbabd	5 days ago	50MB

Рисунок 2.2 – Локально собранные образы микросервисов

2.4 Swagger-документация микросервисов

Каждый из микросервисов бэкенда независим, что позволяет локально запустить тот или иной сервис. В бэкенде приложений реализован Swagger, который описывает API. Для того чтобы попасть в Swagger, нужно запустить собранный образ и перейти по адресу `http://localhost:<port>/docs`. Это делается с помощью команды из листинга 2.3.

Листинг 2.3 – Локальный запуск контейнера

```
$ docker run -p <port>:8000 -t <microservice>:<version>
```

Swagger для микросервиса «kuznechik_crypto» показан на рисунке 2.3, а для микросервиса «tools» – на рисунке 2.4.

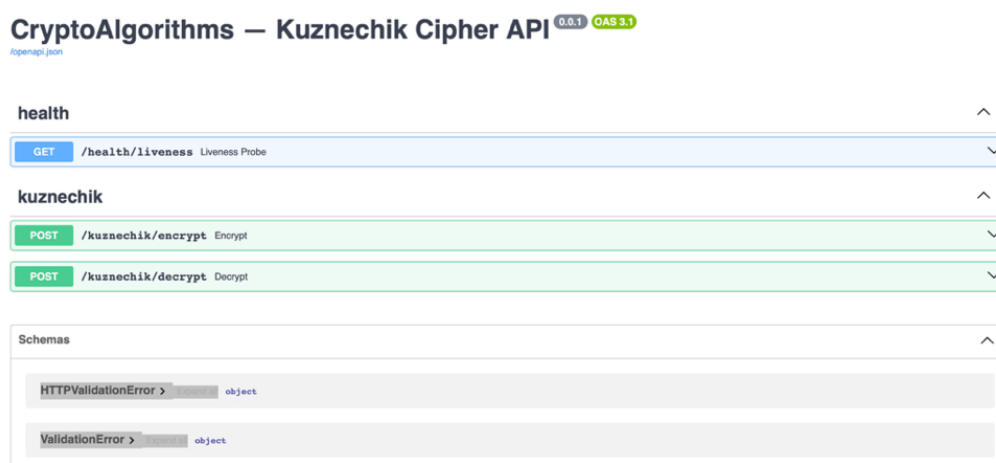


Рисунок 2.3 – Swagger для микросервиса «kuznechik_crypto»

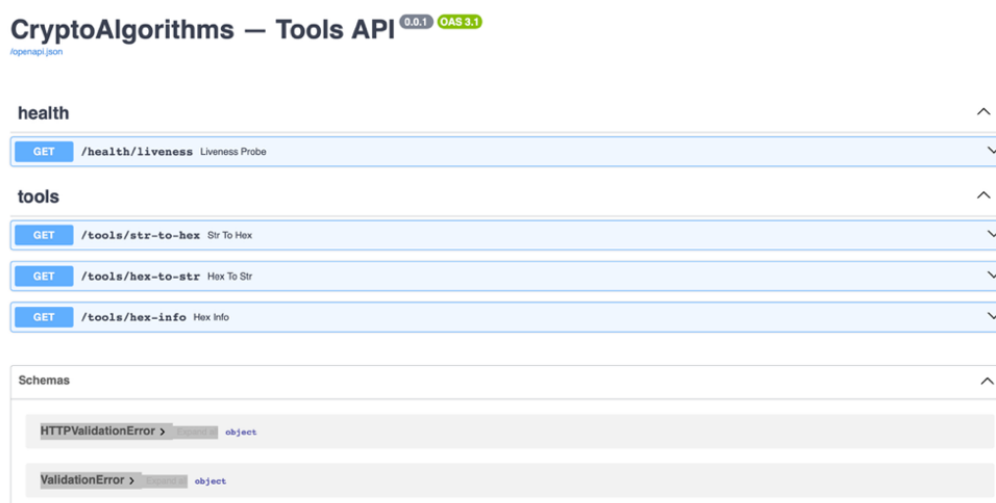


Рисунок 2.4 – Swagger для микросервиса «tools»

Для каждой конечной точки (endpoint) реализована документация, которая позволяет понять, что ожидается на входе, какие есть ограничения и что ожидается на выходе. Так, например, для endpoint kuznechik/encrypt документация показана на рисунке 2.5. Также с помощью Swagger можно взаимодействовать с API, отправив запрос через веб-интерфейс, заполнив требуемые параметры.

POST /kuznechik/encrypt Encrypt

Encrypts a 16-byte block of data using the Kuznechik (GOST R 34.12-2015) block cipher.

This endpoint encrypts a block of data (`blk`) using a provided encryption key (`key`). Both the block and key are represented as hexadecimal strings.

Zero padding:

- If the block of data is less than 16 bytes, zero padding is added to the left side of the byte string to match 16 bytes.
- If the encryption key is less than 32 bytes, zero padding is added to the left side of the byte string to match 32 bytes.
- This ensures compliance with the Kuznechik cipher's requirements for fixed block of data and encryption key sizes.

Parameters:

- blk:** The block of data to be encrypted, represented as a hexadecimal string.
 - Must be no more than 16 bytes (32 hex characters). Zero padding will be applied if shorter.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even.
- key:** The encryption key, represented as a hexadecimal string.
 - Must be no more than 32 bytes (64 hex characters). Zero padding will be applied if shorter.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even.

Returns:

- A hexadecimal string representing the encrypted block of exactly 16 bytes (32 hex characters).

Raises:

- 400 Bad Request:**
 - If the `blk` or `key` does not have an even number of characters.
- 422 Unprocessable Entity:**
 - If the input parameters do not meet the validation requirements.

Example:

```

Input:
blk: "48656c66f"
key: "576f726c64"

Output:
"8479704f8d801853d314e7e060f67a80"

Explanation:
- Block of data is zero padded to "00000000000000000000000048656c66f".
- Encryption key is zero padded to "00000000000000000000000000000000000000000000576f726c64".
- Resulting encrypted block is returned as a hexadecimal string.

```

Parameters

Name	Description
blk required string (query) maxLength: 32 pattern: "[0-9a-fA-F]+"	blk
key required string (query) maxLength: 64 pattern: "[0-9a-fA-F]+"	key

Execute

Cancel

Рисунок 2.5 – Документация и параметры запроса для endpoint kuznechik/encrypt

2.5 Валидация данных

Валидация входных параметров обеспечивает отказоустойчивость и повышает защищенность приложения. Все значения, выходящие за пределы допустимого, автоматически отклоняются, что минимизирует вероятность ошибок и ограничивает возможные векторы атак.

Для каждой конечной точки предусмотрена валидация данных. В таблице 2.1 отражены ожидаемые значения входных параметров конечных точек для двух микросервисов бэкенда.

Таблица 2.1 – Endpoints и ожидаемые значения входных параметров для микросервисов бэкенда

Префикс	Endpoint	Параметр	Ожидаемые значения
/health	/liveness		

41

Продолжение таблицы 2.1

Префикс	Endpoint	Параметр	Ожидаемые значения
/tools	/str-to-hex	data_str	– Строка в формате UTF-8 – Длина не должна превышать 32 символа
	/hex-to-str	data_hex	– Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной – Длина не должна превышать 128 символов
	/hex-info	data_hex	– Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной – Длина не должна превышать 128 символов
/kuznechik	/encrypt	blk	– Длина не должна превышать 16 байт (32 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной
		key	– Длина не должна превышать 32 байта (64 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной
	/decrypt	blk	– Длина ровно 16 байт (32 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F]
		key	– Длина не должна превышать 32 байта (64 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной

В случае, если в endpoint будут переданы данные, которые там не ожидаются, они будут проигнорированы и пользователю будет возвращена ошибка 400 или 422 с соответствующим пояснением, что произошло.

На рисунке 2.6 показана ошибка валидации. Пользователь пытается отправить в микросервис «tools» в endpoint /tools/hex-to-str данные Hello, что недопустимо для данной конечной точки.

```
curl -i 'http://localhost:9000/tools/hex-to-str?data_hex=Hello'
HTTP/1.1 422 Unprocessable Entity
date: Sat, 20 Dec 2025 16:24:56 GMT
server: uvicorn
content-length: 178
content-type: application/json

{"detail":[{"type":"string_pattern_mismatch","loc":["query","data_hex"],"msg":"String should match pattern '^([0-9a-fA-F])+$',\"input\":\"Hello\",\"ctx\":{\"pattern\":\"^([0-9a-fA-F])+$\"}}]}
```

Рисунок 2.6 – Ошибка валидации в endpoint /tools/hex-to-str

На рисунке 2.7 показана та же ошибка, но уже в Swagger.

GET /tools/hex-to-str Hex To Str

Converts a hex string to a regular string.
This endpoint takes a hexadecimal string as input and converts it to a regular UTF-8 string.

Parameters:

- data_hex**: The hexadecimal string to be converted.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even and not exceed 128 characters.

Returns:

- A JSON object with the key `data_str`, containing the decoded string.

Raises:

- 400 Bad Request**:
 - If the hex string does not have an even number of characters.
 - If the hex string cannot be decoded into valid UTF-8 encoded text.
- 422 Unprocessable Entity**:
 - If the input hex string exceeds the maximum allowed length (128 characters).
 - If the input hex string contains invalid hexadecimal characters (the string must only contain characters from `[0-9a-fA-F]`).

Example:

```
Input: "48656c6c66"
Output: {"data_str": "Hello"}
```

Parameters

Name	Description
data_hex * required string (query)	Hello maxlength: 128 pattern: '^([0-9a-fA-F])+\$'

Please correct the following validation errors and try again.

- For 'data_hex': Value must follow pattern '^([0-9a-fA-F])+\$'.

Execute Clear

Рисунок 2.7 – Ошибка валидации в endpoint /tools/hex-to-str, продемонстрированная в Swagger

На рисунке 2.8 показан корректный запрос в endpoint /tools/hex-to-str в Swagger. Пользователь отправляет шестнадцатеричные данные 48656c6c66f, которые преобразуются в строку UTF-8 со значением Hello.

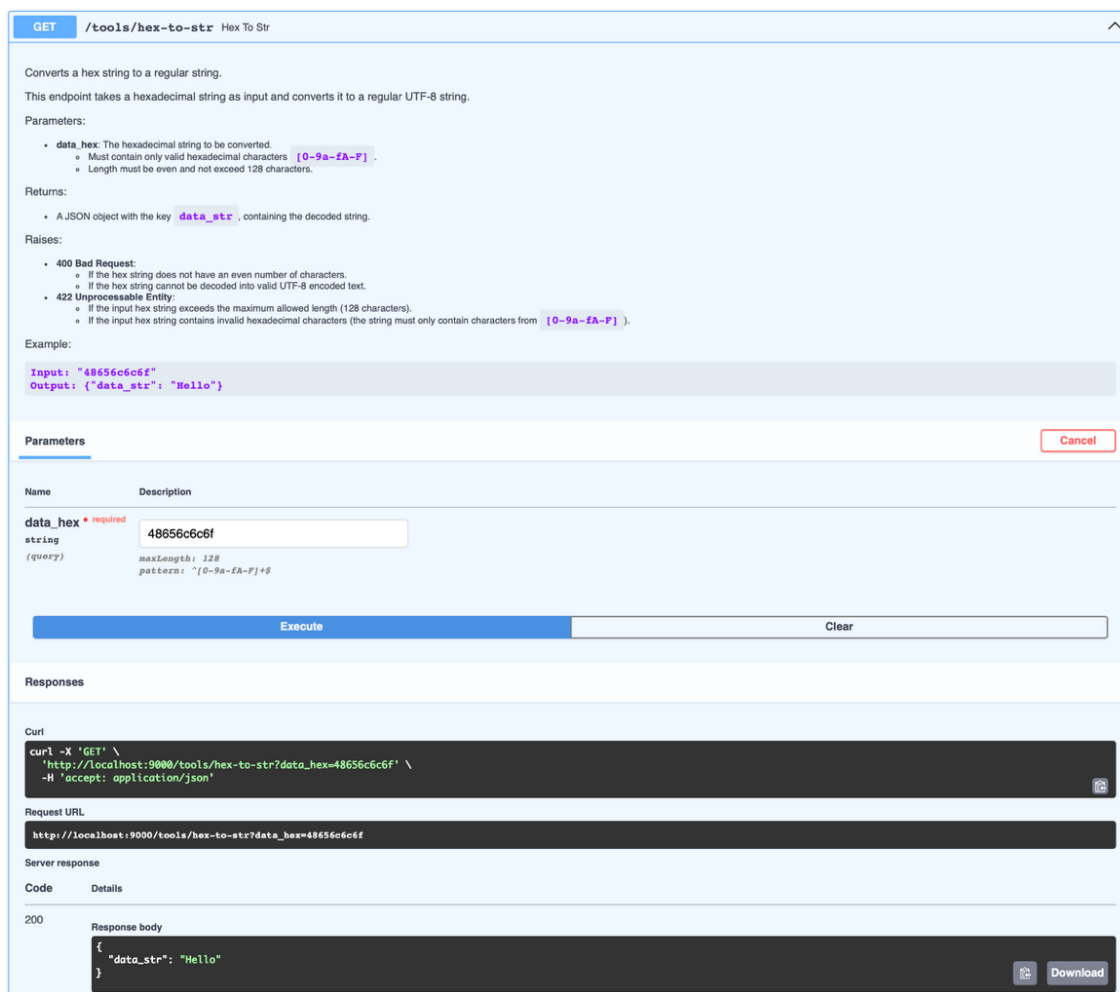


Рисунок 2.8 – Корректный запрос в endpoint /tools/hex-to-str, продемонстрированный в Swagger

2.6 Дополнение нулями для алгоритма шифрования «Кузнечик»

Алгоритм «Кузнечик» работает с длиной шифруемого/дешифруемого блока 128 бит (32 шестнадцатеричных числа или 16 байт) и ключом шифрования/дешифрования 256 бит (64 шестнадцатеричных числа или 32 байта). Но не всегда данные соответствуют необходимым размерам. Для случая, когда размер данных меньше требуемого, предусмотрено дополнение нулями (zero padding).

Zero padding дополняет данные нулевыми байтами слева до нужного размера. Например, пользователь хочет зашифровать слово Hello с помощью ключа World. Представление в шестнадцатеричном формате и длина данных слов показаны в таблице 2.2.

Таблица 2.2 – Информация по словам Hello и World

Слово	Шестнадцатеричное представление	Длина в битах	Длина в шестнадцатеричном формате	Длина в байтах
Hello	48656c6c6f	40	10	5
World	576f726c64	40	10	5

Длина блока данных 40 бит, что меньше требуемых 128, как и длина ключа 40, что меньше требуемых 256 бит. Для решения данной проблемы применяется zero padding, который дополняет шестнадцатеричное представление блока данных и ключа нулевыми байтами слева, до необходимых размеров. Таким образом на вход функции шифрования подаются данные нужного размера. Пример подаваемых данных для слов Hello и World показан в таблице 2.3. В результате шифрования будет получен зашифрованный блок 8479704f8d801853d314e7e060f67a80.

Таблица 2.3 – Информация по словам Hello и World, дополненные нулевыми байтами

Слово	Шестнадцатеричное представление до дополнения нулевыми байтами	Шестнадцатеричное представление после дополнения нулевыми байтами
Hello	48656c6c6f	00000000000000000000000048656c6c6f
World	576f726c64	00000000000000000000000000000000 000000000000000000000000576f726c64

Пример обратной ситуации, пользователь хочет дешифровать зашифрованный блок 8479704f8d801853d314e7e060f67a80. На выходе функции шифрования получается зашифрованный блок длиной 128 бит, поэтому при дешифровании zero padding к зашифрованному блоку не применяется, потому что он уже нужного размера. Если же подается блок меньшего размера, то такой блок не пройдет валидацию и пользователю будет возвращена ошибка 422. Для ключа же zero padding применяется по такому же принципу, как и при шифровании. После дешифрования из полученного блока удаляется zero padding (слева удаляются дополненные нулевые байты), и пользователь получает блок данных, который был зашифрован. Таким образом, при дешифровании блока 8479704f8d801853d314e7e060f67a80

Продолжение таблицы 2.4

Префикс	Endpoint	Краткое описание endpoint
/kuznechik	/decrypt	Расшифровывает 16-байтовый зашифрованный блок с использованием блочного шифра «Кузнечик» (ГОСТ Р 34.12-2015). Этот endpoint расшифровывает зашифрованный блок (blk) с использованием предоставленного ключа для расшифрования (key). Блок и ключ представлены в виде шестнадцатеричных строк.

В таблице 2.5 отображена информация о применении zero padding к входным параметрам endpoints шифрования и дешифрования микросервиса «kuznechik_crypto».

Таблица 2.5 – Применение zero padding в endpoints шифрования и дешифрования микросервиса «kuznechik_crypto»

Префикс	Endpoint	Параметр	Zero padding
/kuznechik	/encrypt	blk	Да
		key	Да
	/decrypt	blk	Нет
		key	Да

Также важно помнить о том, что результат работы endpoint /kuznechik/decrypt возвращается с удаленным zero padding.

2.8 CI-конвейер в Jenkins

В проекте реализован Jenkins пайплайн, позволяющий оптимизировать рутинные задачи в части CI. Пайплайн автоматизирует процессы тестирования кода, проверки кода на безопасность, а также сборку Docker образов и их публикацию в удаленный реестр образов (registry). Схематично пайплайн показан на рисунке 2.9.

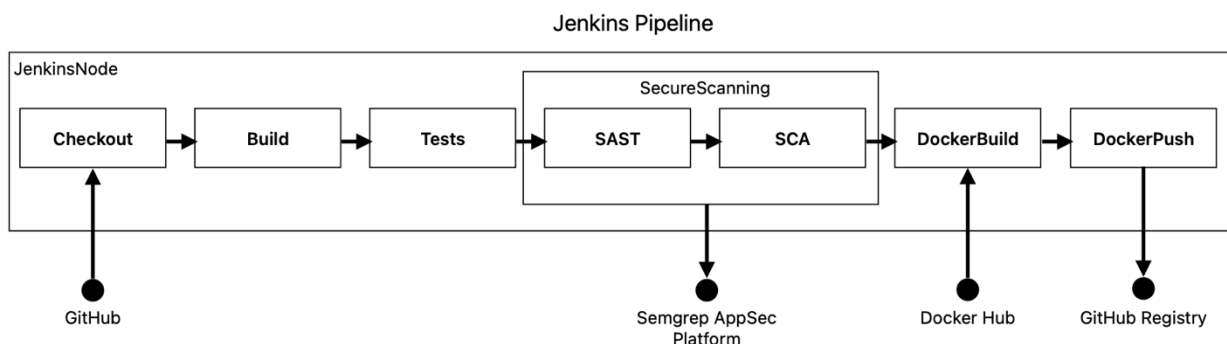


Рисунок 2.9 – Схематичный вид Jenkins пайплайна

На схеме отображены внешние сервисы, с которыми взаимодействует конвейер. Так для этапа «Checkout» должен быть доступ к репозиторию GitHub, для этапа «SecureScanning» должно быть настроено взаимодействие с платформой Semgrep AppSec, для этапа «DockerBuild» требуется доступ к Docker Hub, а для публикации собранных образов, то есть для этапа «DockerPush», должен быть подготовлен удаленный реестр образов, в данном случае это GitHub Registry.

Безопасность ПО проверяется SAST и SCA анализаторами при помощи платформы Semgrep AppSec Platform. Страница входа на платформу показана на рисунке 2.10.

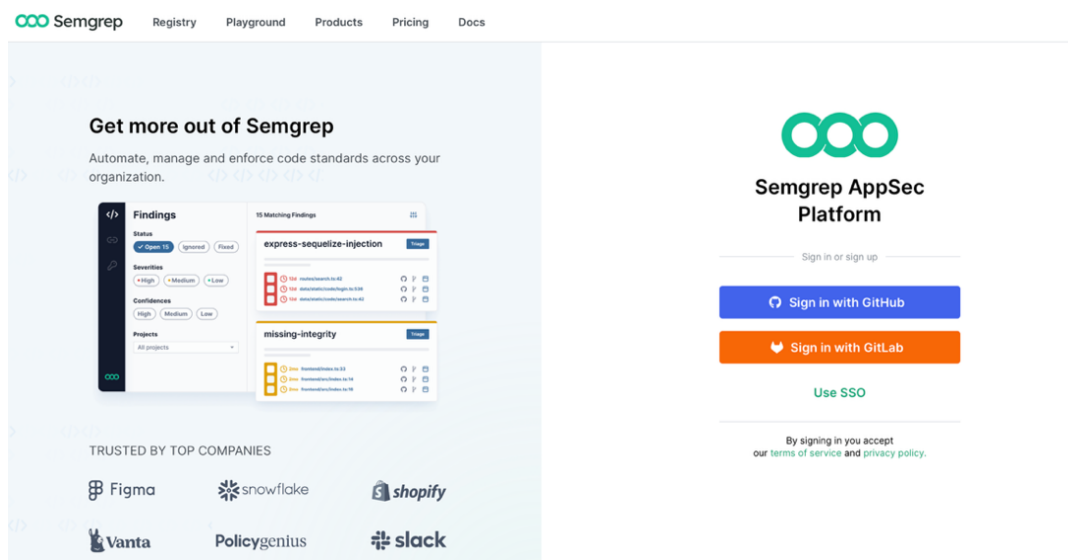


Рисунок 2.10 – Страница входа Semgrep AppSec Platform

Один из критериев хорошего пайплайна – это его адаптивность к изменениям. Добавление новых мультисервисов реализовано крайне просто,

необходимо только указать название нового сервиса. Пример текущий конфигурации мультисервисов, Docker образы которых необходимо собрать и опубликовать, показан в листинге 2.4. Данный конвейер собирает все Docker образы и публикует их параллельно, что позволяет значительно сократить время ожидания.

Листинг 2.4 – Список мультисервисов для сборки и публикации Docker образов

```
SERVICES = "kuznechik_crypto,tools,frontend" // Microservices should be listed separated by ", "
```

Не менее важно иметь возможность управлять запуском работы конвейера. Здесь предусмотрена возможность выбора Jenkins агента, ветки, с которой необходимо собрать проект, можно указать версию контейнеров, которые будут собраны, а также через булевы параметры, можно передать необходимые этапы сборки. Окно запуска пайплайна показано на рисунке 2.11.

The screenshot shows the Jenkins Pipeline Build interface for a pipeline named "Pipeline Build multiservices". It features several input fields and checkboxes for configuring the build. The "jenkins_agent" field is set to "ubuntu", the "branch" field is set to "microservices", and the "container_tag" field is set to "0.0.1". There are three checkboxes for enabling or disabling stages: "secure_scanning" (SAST + SCA), "build_docker" (Build Docker Images for Microservices stage), and "docker_publish" (Publish Docker Images for Microservices stage). The "Build" button is highlighted in green, and the "Cancel" button is in gray.

Pipeline Build multiservices

This build requires parameters:

jenkins_agent
Select the Jenkins agent
ubuntu

branch
Select the branch to build from
microservices

container_tag
Specify the tag for the container to be built
0.0.1

☐ **secure_scanning**
Enable or disable the SecureScanning stage (SAST + SCA)

☐ **build_docker**
Enable or disable the Build Docker Images for Microservices stage

☐ **docker_publish**
Enable or disable the Publish Docker Images for Microservices stage (To enable, the Build Docker Images for Microservices stage must be enabled)

Build **Cancel**

Рисунок 2.11 – Окно запуска пайплайна с выбором необходимых параметров

При каждом запуске всегда будут работать этапы «Checkout», «Build kuznechik_crypto (C++)» и «Test». Все остальные этапы включаются через

булевы параметры. На рисунке 2.12 показана успешная работа конвейера со всеми включенными этапами.

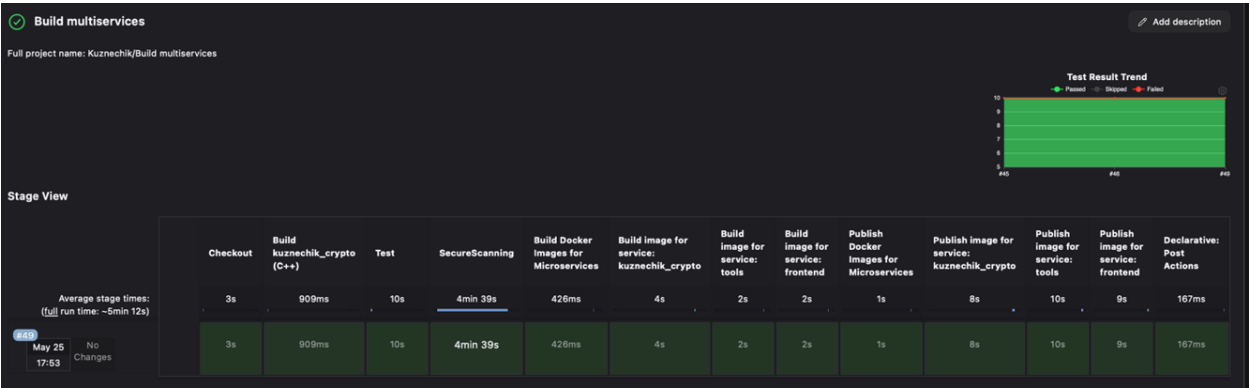


Рисунок 2.12 – Отображение stages в Jenkins UI

Результат сканирования кода на уязвимости, можно посмотреть из логов работы конвейера или в веб-интерфейсе Sengrep AppSec Platform. Опубликованные Docker образы в GitHub Registry показаны на рисунке 2.13. Здесь же можно посмотреть статистику скачивания образов.

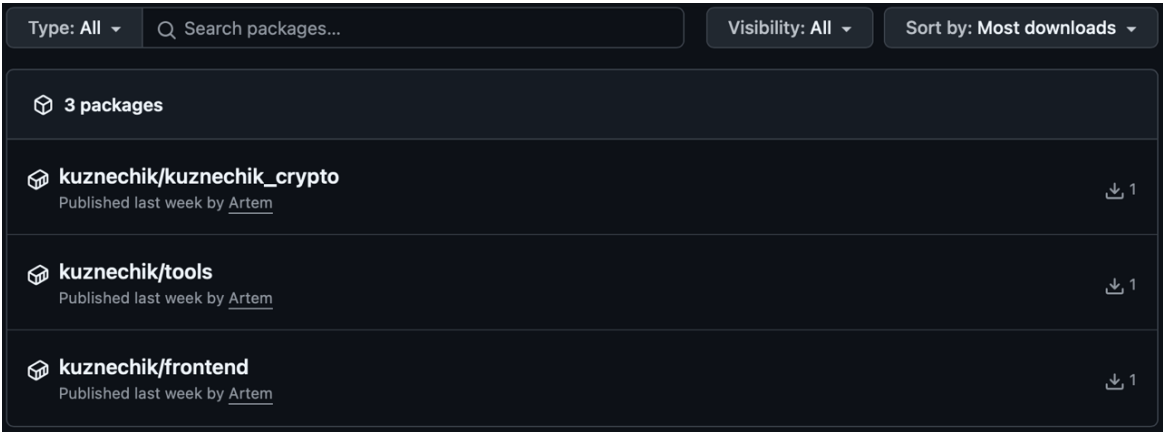


Рисунок 2.13 – Опубликованные Docker образы в GitHub Registry

2.9 Kubernetes манифесты

Для каждого микросервиса были разработаны Kubernetes-манифесты: Deployment и Service. В манифестах определены различные параметры, например стратегия развертывания, количество реплик, настройки проверок работоспособности и типы сервисов. Эти параметры напрямую влияют на стабильность работы компонентов и являются важной частью обеспечения отказоустойчивости микросервисной архитектуры.

Для микросервиса frontend выбрана стратегия развертывания RollingUpdate (листинг 2.5), что позволяет обновлять поды поэтапно, без остановки всего сервиса. Такой подход позволяет обновлять пользовательский интерфейс без недоступности.

Листинг 2.5 – Стратегия развертывание микросервиса frontend

```
strategy:
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1
  type: RollingUpdate
```

Для бэкенд сервисов применяется стратегия Recreate (листинг 2.6). Данный подход исключает одновременное существование разных версий подов и предотвращает возможные несовместимости.

Листинг 2.6 – Стратегия развертывания микросервисов бэкенда

```
strategy:
  type: Recreate
```

Каждому сервису выделены две реплики, что обеспечивает базовую горизонтальную отказоустойчивость: обе реплики работают параллельно, поддерживая стабильную работу сервиса. В случае сбоя одной из них, трафик переключается на оставшуюся, обеспечивая доступность для пользователя. При этом сбойный под автоматически восстанавливается. Как только он снова станет готов к работе, нагрузка постепенно перераспределится между обеими репликами.

Для всех микросервисов настроены проверки работоспособности livenessProbe, позволяющие Kubernetes автоматически выявлять и перезапускать неработающие поды. В микросервисе frontend проверка организована на основе доступности корневой страницы. Реализация livenessProbe показана в листинге 2.7.

Листинг 2.7 – Настройка livenessProbe для микросервиса frontend

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 1
  successThreshold: 1
  failureThreshold: 3
```

В двух бэкендах для этого выделена специальная конечная точка /health/liveness. В листинге 2.8 показана настройка livenessProbe для бэкендов.

Листинг 2.8 – Настройка livenessProbe для микросервисов бэкенда

```
livenessProbe:
  httpGet:
    path: /health/liveness
    port: 8000
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 1
  successThreshold: 1
  failureThreshold: 3
```

Типы сервисов настроены следующим образом:

- сервис frontend использует тип NodePort, что обеспечивает возможность внешнего обращения к приложению;
- сервисы бэкенда имеют тип ClusterIP, что обеспечивает их доступность только внутри кластера и исключает внешний доступ.

Архитектура приложения показана на рисунке 2.14. Пользователи работают с frontend, который обменивается данными с бэкендом, к которому прямого доступа у пользователей нет. Такая изоляция внутренних сервисов повышает надежность системы, снижая риск сбоев и обеспечивая защиту от несанкционированного доступа.

Архитектура приложения

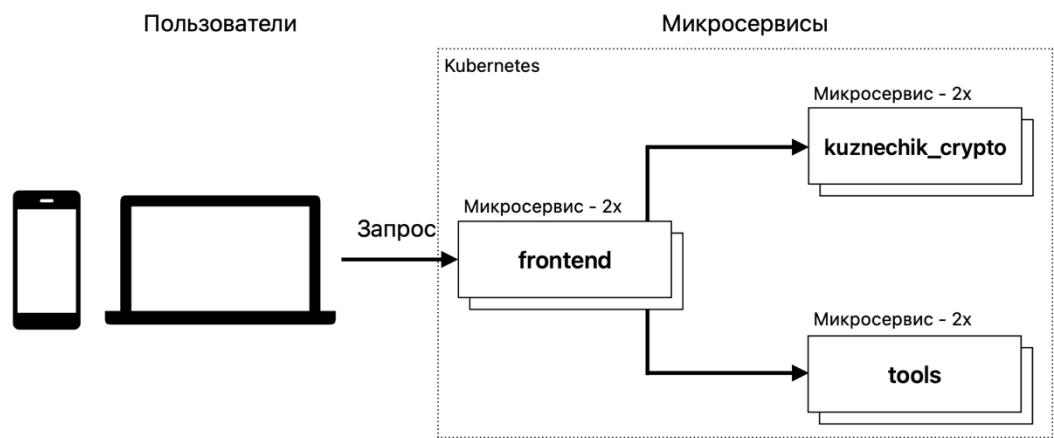


Рисунок 2.14 – Архитектура приложения

Поды, развернутые на основе Kubernetes манифестов показаны на рисунке 2.15.

Pods									
Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑	
frontend-deployment-5584df48bc-dw78b	ghcr.io/rezabungel/kuznechik/frontend:0.0.1	app: frontend component: web pod-template-hash: 5584df48bc	minikube	Running	0	1.00m	7.05Mi	17 minutes ago	⋮
frontend-deployment-5584df48bc-cccqf	ghcr.io/rezabungel/kuznechik/frontend:0.0.1	app: frontend component: web pod-template-hash: 5584df48bc	minikube	Running	0	1.00m	7.09Mi	17 minutes ago	⋮
kuznechik-crypto-deployment-7c49675d45-bzp2l	ghcr.io/rezabungel/kuznechik/kuznechik_crypto:0.1	app: kuznechik-crypto component: api pod-template-hash: 7c49675d45	minikube	Running	0	12.00m	65.05Mi	17 minutes ago	⋮
kuznechik-crypto-deployment-7c49675d45-mpqkd	ghcr.io/rezabungel/kuznechik/kuznechik_crypto:0.1	app: kuznechik-crypto component: api pod-template-hash: 7c49675d45	minikube	Running	0	12.00m	65.13Mi	17 minutes ago	⋮
tools-deployment-9847bccb6-869kq	ghcr.io/rezabungel/kuznechik/tools:0.0.1	app: tools component: api pod-template-hash: 9847bccb6	minikube	Running	0	12.00m	64.49Mi	17 minutes ago	⋮
tools-deployment-9847bccb6-jml95	ghcr.io/rezabungel/kuznechik/tools:0.0.1	app: tools component: api pod-template-hash: 9847bccb6	minikube	Running	0	12.00m	64.51Mi	17 minutes ago	⋮

Рисунок 2.15 – Развернутые поды в Kubernetes

Использование различных стратегий развертывания, нескольких реплик, проверок работоспособности и разграничение типов сервисов обеспечивает отказоустойчивость, снижает время простоя и повышает безопасность микросервисной архитектуры.

Вывод по 2 главе

В данной главе выпускной квалификационной работы была разработана отказоустойчивая информационная система – приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик»), основанное на микросервисной архитектуре.

Здесь представлено описание архитектуры приложения, приведено описание API веб-сервисов бэкендов, продемонстрированы особенности реализации Docker образов микросервисов, а также рассмотрены основные элементы Kubernetes манифестов. Кроме того, были применены лучшие практики при построении CI-конвейера, автоматизирующего процессы сборки и публикации Docker образов.

ГЛАВА 3. ТЕСТИРОВАНИЕ ОТКАЗОУСТОЙЧИВОСТИ ПРОТОТИПА

3.1 Проверка работоспособности

В Kubernetes манифестах определены livenessProbe. Информацию о данной проверке можно найти в логах пода. На рисунке 3.1 показаны логи одного из подов микросервиса «tools».

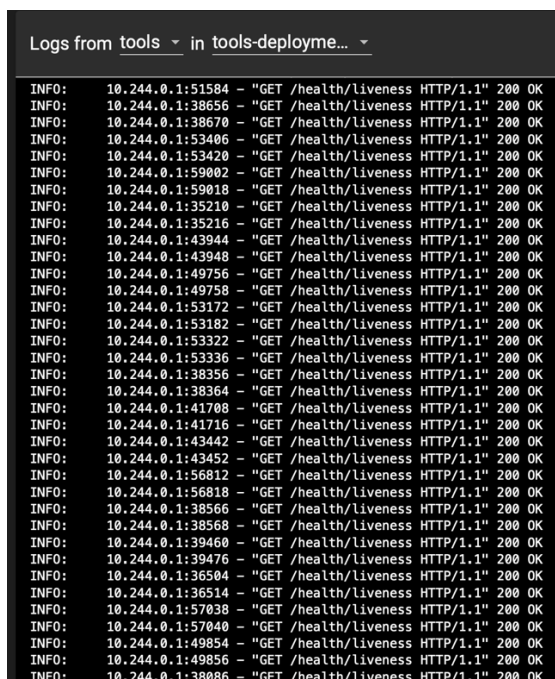


Рисунок 3.1 – Логи одного из подов микросервиса «tools»

Смоделировав ошибку, при которой приложение не может дальше работать, оно, соответственно, не сможет отвечать на проверки. Как показано на рисунке 3.2 проверки отработали неуспешно.

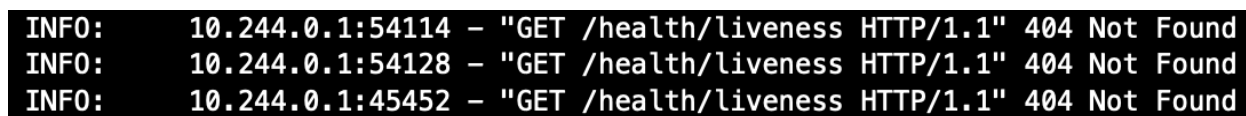


Рисунок 3.2 – Неуспешно отработанные проверки работоспособности

Неуспешно отработанные проверки работоспособности сигнализируют Kubernetes, что с приложением что-то не так, оно больше не отвечает. Kubernetes автоматически восстанавливает работу микросервиса, перезапустив его. Информация об этом показана на рисунке 3.3.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-5584df48bc-dw78b	1/1	Running	0	7h1m
frontend-deployment-5584df48bc-rccqf	1/1	Running	0	7h1m
kuznechik-crypto-deployment-7c49675d45-bzp2l	1/1	Running	0	7h1m
kuznechik-crypto-deployment-7c49675d45-mpqkd	1/1	Running	0	7h1m
tools-deployment-9847bccb6-jml95	1/1	Running	0	7h1m
tools-deployment-9847bccb6-r5wth	1/1	Running	0	68s
tools-deployment-9847bccb6-r5wth	1/1	Running	1 (3s ago)	94s

Рисунок 3.3 – Автоматический перезапуск пода с микросервисом

Так как запущено два экземпляра микросервиса «tools», конечный пользователь не заметит проблемы выхода из строя одного из них. Kubernetes попытается восстановить состояние системы, тем самым достигается отказоустойчивость системы, при возникновении каких-либо ошибок внутри приложения.

3.2 Недоступность микросервиса

Бывают ситуации, когда приложение не может продолжить свою работу по каким-либо причинам. Проверки работоспособности здесь не помогут, потому что приложение неработоспособно, Kubernetes лишь без конца будет перезапускать его.

Смоделирована ситуация, когда микросервис «kuznechik_crypto» недоступен. Пользователь все еще может продолжить взаимодействовать с системой, но для него доступен только микросервис «tools». Данная ситуация показана на рисунке 3.4.

Рисунок 3.4 – Доступен только микросервис «tools»

В системе, реализованной как монолит, такой возможности нет, и система была бы полностью недоступна. Потеря части функционала информационной системы гораздо предпочтительнее, чем потеря всей системы. Микросервисная архитектура именно за счет разделения приложения на независимые компоненты позволяет минимизировать влияние отказа одного сервиса на работу всей системы, обеспечивая таким образом высокую отказоустойчивость и доступность.

3.3 Масштабирование микросервисов

Не все сервисы находятся под одинаковой нагрузкой: одни востребованы больше, другие – меньше. Микросервисная архитектура позволяет увеличивать количество реплик конкретного сервиса. Это позволяет эффективно распределять ресурсы, горизонтально масштабируя только нужные в данный момент сервисы.

Если, например, большое количество пользователей начнет обращаться к микросервису «kuznechik_crypto», может возникнуть ситуация, что ответы будут возвращаться с высокой задержкой или сервис вовсе перестанет отвечать на запросы. Это негативно влияет на пользовательский опыт взаимодействия с системой. Для предотвращения подобных проблем можно временно увеличить количество реплик этого сервиса, как показано на рисунке 3.5.

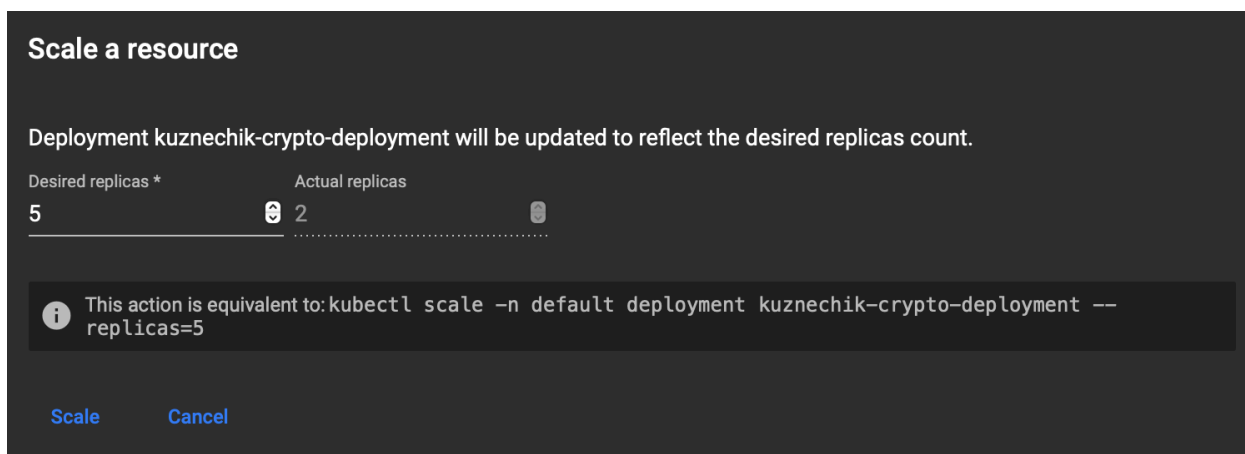


Рисунок 3.5 – Увеличение количества реплик микросервиса «kuznechik_crypto»

Горизонтальное масштабирование поддерживается всеми микросервисами системы, что повышает ее отказоустойчивость, особенно в периоды пиковых нагрузок.

3.4 Обновление системы

Система обладает возможностью обновления микросервиса «frontend» без недоступности. Это достигается путем использования стратегии RollingUpdate. Процесс обновления данного микросервиса показан на рисунке 3.6.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-5584df48bc-zkhwp	1/1	Running	0	9m49s
frontend-deployment-5584df48bc-zrjmg	1/1	Running	0	10m
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Running	0	6m25s
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Running	0	6m25s
tools-deployment-9847bccb6-gbk8h	1/1	Running	0	8m18s
tools-deployment-9847bccb6-wnzms	1/1	Running	0	8m18s
frontend-deployment-77689c6775-4ptvw	0/1	Pending	0	0s
frontend-deployment-5584df48bc-zkhwp	1/1	Terminating	0	13m
frontend-deployment-77689c6775-4ptvw	0/1	Pending	0	0s
frontend-deployment-77689c6775-4ptvw	0/1	ContainerCreating	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	Pending	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	Pending	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	ContainerCreating	0	0s
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-77689c6775-4ptvw	1/1	Running	0	3s
frontend-deployment-77689c6775-wxqh4	1/1	Running	0	5s
frontend-deployment-5584df48bc-zrjmg	1/1	Terminating	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m

Рисунок 3.6 – Обновление микросервиса «frontend» без недоступности

Микросервисы бэкенда обновляются по стратегии Recreate, при которой старые поды сначала удаляются, а затем создаются новые. Данный процесс для микросервиса «kuznechik_crypto» показан на рисунке 3.7. Такой подход не позволяет провести обновление без недоступности, но он исключает одновременное существование разных версий подов и предотвращает возможные несовместимости.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-77689c6775-4ptvw	1/1	Running	0	9m16s
frontend-deployment-77689c6775-wxqh4	1/1	Running	0	9m16s
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Running	0	19m
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Running	0	19m
tools-deployment-9847bccb6-gbk8h	1/1	Running	0	21m
tools-deployment-9847bccb6-wnzms	1/1	Running	0	21m
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Terminating	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Terminating	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	ContainerCreating	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	ContainerCreating	0	0s
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-5cff4b89c4-bvstz	1/1	Running	0	4s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	1/1	Running	0	6s

Рисунок 3.7 – Обновление микросервиса «kuznechik_crypto» по стратегии Recreate

Выбор стратегии RollingUpdate для обновления микросервиса «frontend» обеспечивает плавную замену его экземпляров без остановки работы, что гарантирует доступность пользователя к UI части сервиса. Это снижает риски простоев и улучшает пользовательский опыт за счет непрерывной работы интерфейса. В то же время, для микросервисов бэкенда применяется стратегия Recreate, которая гарантирует стабильность и согласованность данных, несмотря на временную недоступность. Такой подход повышает общую отказоустойчивость системы, сочетая непрерывность обслуживания и надежность.

Вывод по 3 главе

В данной главе выпускной квалификационной работы была протестирована разработанная информационная система. Результаты тестирования показывают, что применение микросервисной архитектуры и использование инструментов Kubernetes позволяют эффективно обеспечивать отказоустойчивость системы. Автоматическое обнаружение сбоев с помощью проверок работоспособности (livenessProbe) и их автоматическое устранение посредством перезапуска подов предотвращают длительные простои

сервисов. Горизонтальное масштабирование микросервисов в периоды повышенной нагрузки позволяет поддерживать стабильную производительность и минимизировать задержки в ответах. Применение стратегии RollingUpdate для обновления микросервиса «frontend» обеспечивает непрерывную работу интерфейса без простоев, улучшая пользовательский опыт. В то же время стратегия Recreate, используемая для бэкенд сервисов, гарантирует согласованность данных, что повышает надежность системы.

Таким образом, в ходе тестирования продемонстрирована высокая эффективность современных архитектурных решений обеспечения отказоустойчивости на базе микросервисов и оркестрации контейнеров.

ЗАКЛЮЧЕНИЕ

Актуальность темы обеспечения отказоустойчивости в современных информационных системах обусловлена растущими требованиями к стабильности, доступности и непрерывности цифровых сервисов. Сбои в работе компонентов ИТ-инфраструктуры могут приводить к серьезным последствиям: потерям данных, снижению доверия пользователей и финансовым убыткам. В условиях постоянно увеличивающейся нагрузки и усложняющейся архитектуры приложений особенно важным становится использование современных подходов к построению отказоустойчивых систем.

Целью выпускной квалификационной работы была разработка отказоустойчивой информационной системы, основанной на микросервисной архитектуре, с применением лучших практик обеспечения отказоустойчивости.

В ходе выполнения дипломной работы были получены следующие результаты.

1. Проведен анализ актуальности микросервисной архитектуры для построения отказоустойчивых информационных систем, который показал, что данный подход является эффективным и современным решением для цифровых платформ и объектов критической информационной инфраструктуры, обеспечивающий масштабируемость, гибкость и устойчивость к сбоям за счет независимых сервисов.

2. Выполнен сравнительный анализ инструментов обеспечения отказоустойчивости микросервисных систем, в рамках которого были рассмотрены инструменты контейнеризации и оркестрации, показавший, что несмотря на общую схожесть функциональности в каждой категории, существуют различия в архитектуре, подходах к безопасности, уровнях автоматизации и доступной поддержке, поэтому выбор конкретного инструмента определяется задачами и потребностями проекта.

3. Проанализирована разработка архитектуры отказоустойчивой микросервисной информационной системы, которая выявила ключевые принципы проектирования надежной и устойчивой к сбоям системы, что обеспечивает формирование архитектуры, закладывающей основу для стабильной работы и упрощения сопровождения и масштабирования компонентов.

4. Реализован прототип отказоустойчивой микросервисной системы, который представляет собой приложение на микросервисной архитектуре с применением современных архитектурных решений обеспечения отказоустойчивости на базе микросервисов и оркестрации контейнеров средствами Kubernetes. Система включает микросервисы пользовательского интерфейса, бэкендов и CI-конвейер, обеспечивает горизонтальное масштабирование, автоматическое восстановление после сбоев и безопасное обновление компонентов.

5. Проведено тестирование отказоустойчивости прототипа, показавшее его устойчивость к отказам отдельных компонентов, способность к автоматическому восстановлению, поддержку горизонтального масштабирования микросервисов, а также непрерывную доступность пользовательского интерфейса при обновлении. Эти результаты подтверждают целесообразность использования микросервисного подхода, Docker контейнеризации и оркестрации Kubernetes для построения отказоустойчивых информационных систем.

Результатом проделанной работы является микросервисное приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик») на C++ и взаимодействием по REST API на Python.

Дальнейшее развитие системы предполагается за счет внедрения мониторинга и алертинга на базе Prometheus и Grafana, что позволит отслеживать ключевые метрики производительности, доступности и ошибок в работе микросервисов. Настраиваемые дашборды обеспечат визуализацию состояния системы, а алерты – своевременное информирование о сбоях. Это

повысит устойчивость системы, упростит диагностику инцидентов и ускорит реагирование на них.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое информационная безопасность и какие данные она охраняет [электронный ресурс] – URL: <https://cloud.vk.com/blog/informacionnaya-bezopasnost-i-kakie-dannye-ona-ohranyayet/> (дата обращения 26.10.2025).
2. Отказоустойчивая IT-инфраструктура: принципы и способы организации [электронный ресурс] – URL: <https://timeweb.cloud/tutorials/servers/otkazoustojchivaya-it-infrastruktura> (дата обращения 26.10.2025).
3. Что такое микросервисная архитектура и когда ее использовать [электронный ресурс] – URL: <https://atwinta.ru/material/blog/chto-takoe-mikroservisnaya-arhitektura-i-kogda-ee-ispolzovat/> (дата обращения 26.10.2025).
4. Соглашение об уровне обслуживания (SLA) [электронный ресурс] – URL: <https://yandex.cloud/ru/docs/glossary/sla> (дата обращения 26.10.2025).
5. Как повысить отказоустойчивость ИТ-инфраструктуры [электронный ресурс] – URL: <https://onlanta.ru/press/blog/kak-povysit-otkazoustoychivost-it-infrastruktury/> (дата обращения 26.10.2025).
6. Отказоустойчивые системы: зачем нужны и как построить [электронный ресурс] – URL: <https://habr.com/ru/companies/x-com/articles/697796/> (дата обращения 26.10.2025).
7. RAID [электронный ресурс] – URL: <https://ddos-guard.ru/technologies/raid> (дата обращения 28.10.2025).
8. Описание уровней RAID [электронный ресурс] – URL: <https://docs.itvgroup.ru/confluence/pages/viewpage.action?pageId=224299196> (дата обращения 28.10.2025).
9. RAID [электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/RAID> (дата обращения 28.10.2025).
10. План резервного копирования и аварийного восстановления данных [электронный ресурс] – URL: <https://backupsolution.ru/backup-plan/> (дата обращения 29.10.2025).

11. Бэкап: что такое резервное копирование данных [электронный ресурс] – URL: <https://yandex.cloud/ru/docs/glossary/backup> (дата обращения 29.10.2025).

12. Uptime Institute Releases 2021 Global Data Center Survey [электронный ресурс] – URL: <https://facilityexecutive.com/uptime-institute-releases-2021-global-data-center-survey/> (дата обращения 29.10.2025).

13. Облачные решения для обеспечения отказоустойчивости: как гарантировать непрерывность бизнеса [электронный ресурс] – URL: <https://securitymedia.org/info/oblachnye-resheniya-dlya-obespecheniya-otkazoustoychivosti-kak-garantirovat-nepreryvnost-biznesa.html> (дата обращения 30.10.2025).

14. Отказоустойчивый кластер: что это такое и как работает [электронный ресурс] – URL: <https://www.ispsystem.ru/news/high-availability-cluster> (дата обращения 30.10.2025).

15. Монолитная и микросервисная архитектура. Сравнение [электронный ресурс] – URL: <https://habr.com/ru/companies/haulmont/articles/758780/> (дата обращения 01.11.2025).

16. Чем Docker отличается от виртуальной машины? [электронный ресурс] – URL: <https://wiki.merionet.ru/articles/chem-docker-on-otlichaetsya-ot-virtualnoj-mashiny> (дата обращения 01.11.2025).

17. K8s для начинающих [электронный ресурс] – URL: <https://www.reg.ru/blog/k8s-dlya-nachinayushchih> (дата обращения 01.11.2025).

18. Ускорьте свою базу данных: 7 проверенных методов масштабирования и оптимизации [электронный ресурс] – URL: <https://proglib.io/p/uskorte-svoyu-bazu-dannyh-7-proverennyh-metodov-masshtabirovaniya-i-optimizacii-2024-08-13> (дата обращения 02.11.2025).

19. Что такое DevOps: практики, методология, инструменты и почему бизнесу стоит его внедрять в разработку? [электронный ресурс] – URL:

<https://timeweb.cloud/blog/devops-praktiki-metodologiya-instrumenty> (дата обращения 03.11.2025).

20. Стратегии развертывания в DevOps [электронный ресурс] – URL: <https://mws.ru/blog/strategii-razvertyvaniya-v-devops/> (дата обращения 03.11.2025).

21. Docker или Podman: что лучше подойдет для вашего проекта [электронный ресурс] – URL: <https://proglab.io/p/docker-ili-podman-cto-luchshe-podoydet-dlya-vashego-proekta-2024-07-01> (дата обращения 04.11.2025).

22. OpenShift и Kubernetes: сравнительный обзор, основные различия [электронный ресурс] – URL: <https://habr.com/ru/companies/sberbank/articles/857908/> (дата обращения 04.11.2025).

23. ГОСТ Р 34.12—2015. Информационная технология. Криптографическая защита информации. Блочные шифры : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 19 июня 2015 г. № 749-ст : введен впервые : дата введения 2016-01-01 / разработан Центром защиты информации и специальной связи ФСБ России с участием Открытого акционерного общества «Информационные технологии и коммуникационные системы» (ОАО «ИнфоТеКС»). — Москва : Стандартинформ, 2015. — IV, 21с.